

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



BÁO CÁO PROJECT II
ĐỀ TÀI:
XÂY DỰNG WEBSITE THƯƠNG MẠI ĐIỆN TỬ
MỌNG FRUITBOXZ

Học phần	: Project II
Mã lớp	: 761982
Người thực hiện	: Phan Anh Tài
Mã số sinh viên	: 20230063
Giảng viên hướng dẫn	: Lê Tấn Hùng

HÀ NỘI, 6/2026

Lời Nói Đầu

Thương mại điện tử đang trở thành một kênh bán hàng quan trọng đối với các cửa hàng thực phẩm tươi. Tuy nhiên, đặc thù của trái cây và hộp quà khiến bài toán không chỉ dừng ở việc hiển thị sản phẩm và nhận đơn. Hệ thống còn phải xử lý biến thể, giá bán, khuyến mãi, tồn kho nguyên liệu, giao hàng theo khu vực, nội dung truyền thông và trải nghiệm tư vấn trước khi mua.

Từ nhu cầu đó, đề tài “*Xây dựng website thương mại điện tử Mộng Fruitboxz*” được thực hiện nhằm xây dựng một nền tảng bán trái cây tươi, sản phẩm cắt sẵn và hộp quà. Sản phẩm gồm ba phần chính: giao diện mua sắm dành cho khách hàng, giao diện quản trị dành cho nhân viên cửa hàng và hệ thống dịch vụ backend. Ngoài quy trình duyệt sản phẩm, giỏ hàng và đặt hàng, dự án còn triển khai tìm kiếm toàn văn, blog, quản lý nội dung, phân quyền, chatbot tư vấn và quản lý định mức nguyên liệu.

Về kỹ thuật, hệ thống sử dụng React, Vite và Tailwind CSS cho giao diện; Medusa.js v2 và Node.js cho backend; PostgreSQL, Redis, Meilisearch và MinIO cho hạ tầng dữ liệu. Kiến trúc headless commerce tách phần giao diện khỏi lõi nghiệp vụ, đồng thời tận dụng các mô-đun thương mại điện tử có sẵn của Medusa và mở rộng bằng các mô-đun riêng của cửa hàng.

Báo cáo được chỉnh sửa sau khi rà soát mã nguồn tại thời điểm tháng 6 năm 2026. Nội dung vì vậy tập trung vào những phần đã có trong dự án, các điểm còn thiếu và những việc có thể làm tiếp.

Em xin chân thành cảm ơn giảng viên hướng dẫn, thầy **Lê Tấn Hùng**, đã hỗ trợ và tạo điều kiện trong quá trình thực hiện đề tài.

Trân trọng,

Phan Anh Tài

Mục lục

Lời Nói Đầu	i
1 Giới Thiệu Bài Toán	1
1.1 Bối Cảnh	1
1.2 Mục Tiêu Đề Tài	1
1.3 Phạm Vi Hệ Thống	1
1.3.1 Giao diện khách hàng	2
1.3.2 Giao diện quản trị	2
1.3.3 Backend và hạ tầng	2
1.4 Đối Tượng Sử Dụng	2
1.5 Các Nhóm Chức Năng Chính	2
1.6 Yêu Cầu Phi Chức Năng	3
1.7 Kết Cấu Báo Cáo	3
2 Đặc Tả Yêu Cầu	4
2.1 Mục Đích và Phạm Vi Đặc Tả	4
2.2 Công Nghệ và Ràng Buộc	4
2.3 Danh Sách Use Case	4
2.3.1 Quy ước mô hình hóa	6
2.4 Đặc Tả Các Luồng Nghiệp Vụ Chính	6
2.4.1 UC02 – Tìm kiếm sản phẩm	6
2.4.2 UC03 – Quản lý giỏ hàng	7
2.4.3 UC05 – Đặt hàng	7
2.4.4 UC08 – Chatbot tư vấn	9
2.4.5 UC13 – Phân quyền quản trị	9
2.4.6 UC14 – Cấu hình giao hàng và website	10
2.5 Truy Vết Yêu Cầu Sang Thiết Kế	10
2.6 Yêu Cầu Phi Chức Năng	11
2.7 Yêu Cầu Dữ Liệu và Kiểm Tra Hợp Lệ	11
2.8 Tiêu Chí Chấp Nhận	11
2.9 Giới Hạn Phạm Vi Hiện Tại	12
3 Thiết Kế Hệ Thống	13

3.1	Chuyển Từ Phân Tích Sang Thiết Kế	13
3.2	Kiến Trúc Tổng Thể	13
3.3	Thiết Kế Frontend	14
3.3.1	Tổ chức ứng dụng	14
3.3.2	Nguyên tắc giao tiếp API	14
3.4	Thiết Kế Backend Medusa	14
3.4.1	Mô-đun lõi và mô-đun mở rộng	14
3.4.2	API, workflow và subscriber	15
3.5	Luồng Checkout	15
3.5.1	Biểu đồ hoạt động	16
3.5.2	Định vị và tự động báo phí giao hàng	17
3.5.3	Sơ đồ luồng xử lý đặt hàng	18
3.6	Mô Hình Lớp Miền	19
3.7	Tìm Kiếm và Chatbot	19
3.8	Bảo Mật	20
3.9	Vòng Đời Đơn Hàng	21
3.10	Mô Hình Triển Khai	21
4	Thiết Kế Dữ Liệu	23
4.1	Tổng Quan	23
4.2	Nhóm Thực Thể Lõi Của Medusa	23
4.3	Các Model Tùy Biến	24
4.3.1	Mô-đun Site	24
4.3.2	Mô-đun RBAC	24
4.3.3	Mô-đun Ingredients	25
4.3.4	Các mô-đun khác	25
4.4	Quan Hệ Dữ Liệu	26
4.5	Ánh Xạ Mô Hình Miền Sang Lưu Trữ	26
4.6	Nhất Quán Giao Dịch	27
4.7	Dữ Liệu Phi Quan Hệ	27
4.7.1	Redis	27
4.7.2	Meilisearch	27
4.7.3	pgvector	27
4.7.4	MinIO	27
4.8	Toàn Vẹn, Sao Lưu và Bảo Mật Dữ Liệu	27
5	Thiết Kế Giao Diện	29
5.1	Định Hướng Thiết Kế	29
5.2	Sơ Đồ Điều Hướng Frontend	29

5.3	Các Màn Hình Khách Hàng	29
5.4	Ảnh Chụp Giao Diện Từ Phiên Chạy Thử	30
5.4.1	Trang sản phẩm và chi tiết	31
5.4.2	Giỏ hàng và checkout	32
5.4.3	Chatbot	32
5.5	Khu Vực Quản Trị	32
5.6	Nhận Xét Về Tính Nhất Quán	34
6	Tổng Kết	35
6.1	Kết Quả Triển Khai	35
6.2	Đối Chiếu Kết Quả Theo Nhóm Chức Năng	35
6.3	Những Điểm Cần Hoàn Thiện	36
6.4	Hướng Phát Triển	37
6.5	Kiến Thức và Kinh Nghiệm Thu Được	37
6.6	Kết Luận	38
6.7	Tài Liệu Tham Khảo	38

Chương 1: Giới Thiệu Bài Toán

1.1 Bối Cảnh

Mong Fruitboxz là mô hình cửa hàng kinh doanh trái cây tươi, trái cây cắt sẵn và hộp quà. Với mô hình này, khách hàng cần xem được sản phẩm theo nhu cầu, lựa chọn biến thể, biết rõ giá và phí giao hàng trước khi đặt mua. Ở phía cửa hàng, nhân viên cần một công cụ thống nhất để quản lý sản phẩm, đơn hàng, khách hàng, tồn kho, nội dung truyền thông và các chương trình khuyến mãi.

Nếu các nghiệp vụ trên được xử lý bằng nhiều công cụ rời rạc, dữ liệu giá, tồn kho và đơn hàng dễ mất đồng bộ. Việc tìm kiếm sản phẩm, theo dõi hiệu quả kinh doanh hoặc kiểm soát quyền thao tác của nhân viên cũng trở nên khó khăn. Do đó, đề tài hướng tới một nền tảng web tập trung nhưng vẫn có kiến trúc mô-đun để có thể mở rộng theo quy mô kinh doanh.

1.2 Mục Tiêu Đề Tài

Mục tiêu tổng quát là xây dựng một website thương mại điện tử có thể phục vụ trọn vẹn hành trình từ khám phá sản phẩm đến ghi nhận đơn hàng, đồng thời cung cấp công cụ vận hành cho cửa hàng. Các mục tiêu cụ thể gồm:

- Xây dựng catalog sản phẩm, danh mục, trang chi tiết, tìm kiếm và lọc sản phẩm.
- Hỗ trợ giỏ hàng, mã khuyến mãi, tính phí giao hàng và đặt hàng cho cả khách vãng lai lẫn khách đã đăng nhập.
- Cung cấp tài khoản khách hàng và khả năng tra cứu lịch sử đơn hàng.
- Xây dựng khu vực quản trị cho sản phẩm, danh mục, đơn hàng, khách hàng, khuyến mãi, tồn kho, nội dung và người dùng nội bộ.
- Quản lý tồn kho theo nguyên liệu và công thức cấu thành sản phẩm, phù hợp với sản phẩm trái cây cắt sẵn hoặc hộp phối sẵn.
- Tích hợp tìm kiếm toàn văn, lưu trữ ảnh, bộ nhớ đệm và chatbot tư vấn có cơ chế dự phòng.
- Tách biệt rõ giao diện, API, nghiệp vụ và hạ tầng để thuận tiện bảo trì và triển khai.

1.3 Phạm Vi Hệ Thống

1.3.1 Giao diện khách hàng

Phần frontend cung cấp trang chủ, danh sách và chi tiết sản phẩm, danh mục, tìm kiếm, giỏ hàng, checkout, đăng ký/đăng nhập, tài khoản, lịch sử đơn, blog, liên hệ và các trang chính sách. Hệ thống còn có chức năng *Hộp tự chọn*, cho phép khách xây dựng hộp trái cây theo nhu cầu.

1.3.2 Giao diện quản trị

Khu vực quản trị là một ứng dụng React trong cùng frontend, được bảo vệ bằng JWT và quyền RBAC. Nhân viên có thể truy cập dashboard, báo cáo tài chính, sản phẩm, danh mục, khách hàng, đơn hàng, khuyến mãi, kho, nguyên liệu, banner, blog, media, chatbot, cấu hình giao hàng và nội dung chính sách tùy theo quyền được cấp.

1.3.3 Backend và hạ tầng

Backend Medusa v2 cung cấp các mô-đun thương mại điện tử lõi và các API tùy biến. PostgreSQL lưu dữ liệu giao dịch; Redis hỗ trợ cache, event bus, rate limit và giỏ hàng phiên; Meiliseach lập chỉ mục tìm kiếm; MinIO lưu ảnh và tệp media. Các dịch vụ hạ tầng được mô tả bằng Docker Compose để thuận tiện khởi tạo môi trường phát triển.

1.4 Đối Tượng Sử Dụng

Tác nhân	Quyền và nhu cầu chính
Khách vãng lai	Duyệt và tìm kiếm sản phẩm, quản lý giỏ hàng, nhận báo giá giao hàng, đặt đơn khách, đọc blog và hỏi chatbot.
Khách hàng	Có thêm khả năng quản lý tài khoản và xem lịch sử đơn hàng.
Biên tập viên	Quản lý banner, bài viết, media, nội dung chính sách và FAQ/chatbot khi được cấp quyền nội dung tương ứng.
Quản lý cửa hàng	Vận hành catalog, đơn hàng, khách hàng, khuyến mãi, tồn kho, nguyên liệu và báo cáo kinh doanh.
Super Admin	Có toàn quyền hệ thống, đặc biệt là quản lý người dùng nội bộ, vai trò, quyền hạn, cấu hình giao hàng và các thiết lập quan trọng.
Dịch vụ ngoài	Meiliseach phục vụ tìm kiếm; MinIO lưu media; Nominatim hỗ trợ địa chỉ; Groq cung cấp mô hình ngôn ngữ cho chatbot.

1.5 Các Nhóm Chức Năng Chính

- Catalog và nội dung:** sản phẩm, biến thể, danh mục, banner, blog và trang chính sách.
- Bán hàng:** giỏ hàng, khuyến mãi, phí giao hàng, checkout, mã đơn và lịch sử đơn.

3. **Tương tác khách hàng:** đăng ký, đăng nhập, liên hệ và chatbot tư vấn.
4. **Vận hành:** quản lý đơn, khách hàng, tồn kho, công thức nguyên liệu và dashboard tài chính.
5. **Quản trị hệ thống:** người dùng nội bộ, vai trò, quyền, cấu hình website, media và trạng thái tìm kiếm.

1.6 Yêu Cầu Phi Chức Năng

- **Bảo mật:** xác thực bằng JWT, phân quyền ở cả giao diện và API, cấu hình CORS, giới hạn tần suất chatbot và không tin cậy giá tiền gửi từ trình duyệt.
- **Nhất quán dữ liệu:** backend truy vấn lại giá biến thể, kiểm tra định mức nguyên liệu và áp dụng khuyến mãi trước khi tạo đơn.
- **Khả dụng:** tìm kiếm và giỏ hàng có cơ chế dự phòng khi Meilisearch hoặc Redis chưa sẵn sàng.
- **Hiệu năng:** lazy loading các trang React, cache dữ liệu thường dùng và đồng bộ chỉ mục bằng subscriber theo sự kiện.
- **Khả năng mở rộng:** mô-đun tùy biến được tách khỏi lõi Medusa; hạ tầng lưu trữ, cache và tìm kiếm có thể triển khai độc lập.
- **Khả năng sử dụng:** giao diện responsive, hỗ trợ tiếng Việt và tiếng Anh, có trạng thái loading, empty, error và phản hồi thao tác.

1.7 Kết Cấu Báo Cáo

Chương 2 đặc tả yêu cầu và các luồng nghiệp vụ. Chương 3 trình bày kiến trúc, thành phần và luồng xử lý. Chương 4 mô tả mô hình dữ liệu. Chương 5 phân tích các nhóm giao diện. Chương 6 tổng kết kết quả, giới hạn và hướng phát triển.

Chương 2: Đặc Tả Yêu Cầu

2.1 Mục Đích và Phạm Vi Đặc Tả

Chương này mô tả các yêu cầu được rút ra từ mã nguồn hiện tại, thay vì mô tả một hệ thống giả định. Mỗi yêu cầu được gắn với tác nhân, đầu vào, kết quả và trạng thái triển khai. Nhờ đó, từng phần trong báo cáo có thể đối chiếu lại với giao diện, API và dữ liệu trong mã nguồn.

2.2 Công Nghệ và Ràng Buộc

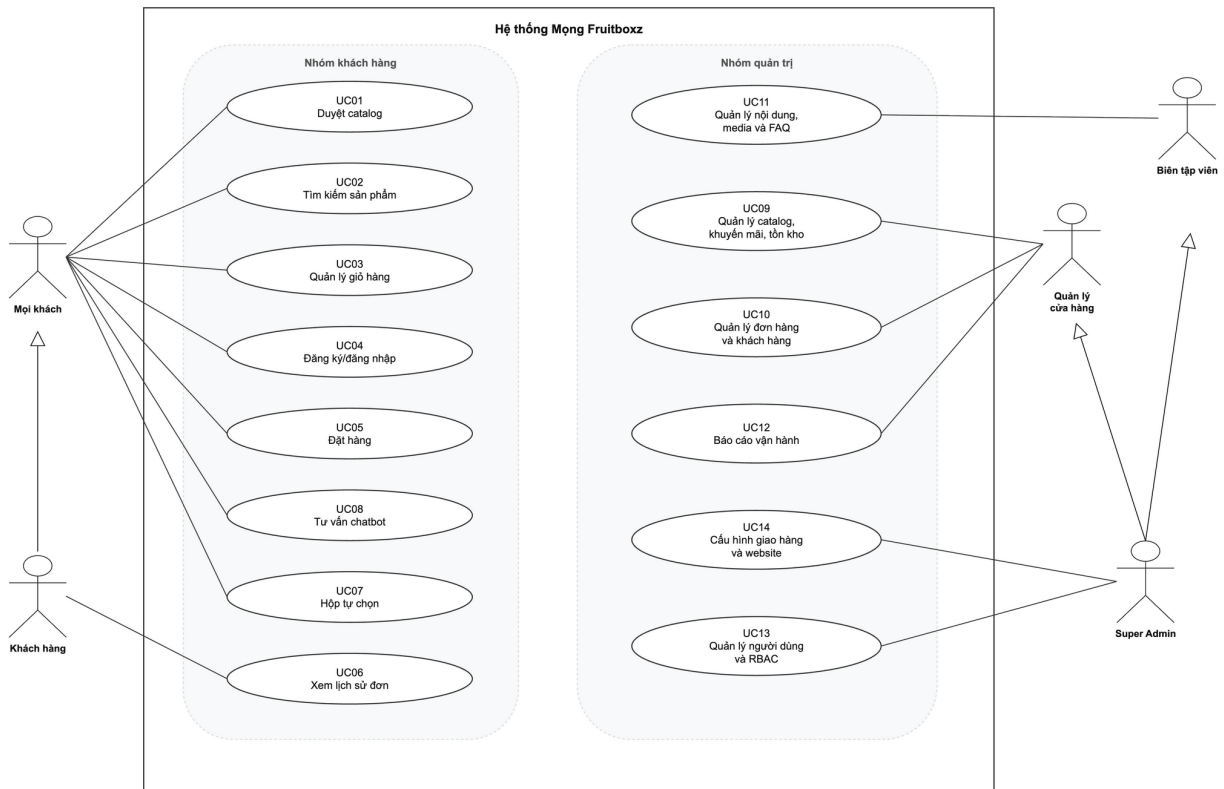
Thành phần	Công nghệ	Vai trò trong dự án
Frontend	React 19, Vite 7, Tailwind CSS 4	Giao diện mua sắm và khu vực quản trị tùy biến.
Backend	Medusa.js 2.15, Node.js 20, TypeScript	API, mô-đun thương mại điện tử, xác thực và nghiệp vụ.
Cơ sở dữ liệu	PostgreSQL 15 + pgvector	Dữ liệu giao dịch, catalog, người dùng, mô-đun tùy biến và hạ tầng vector chuẩn bị cho tìm kiếm/ngữ cảnh ngữ nghĩa.
Cache và sự kiện	Redis 7	Cache, event bus, rate limit và đồng bộ giỏ hàng phiên khi được bật.
Tìm kiếm	Meilisearch 1.13	Lập chỉ mục và tìm kiếm sản phẩm theo từ khóa, danh mục, mức giá.
Lưu trữ đối tượng	MinIO, giao thức S3	Ảnh sản phẩm, banner và media quản trị.
AI tư vấn	Groq API	Sinh câu trả lời từ FAQ và ngữ cảnh sản phẩm, có fallback khi dịch vụ lỗi.
Môi trường	Docker Compose	Khởi tạo PostgreSQL, Redis, Meilisearch và MinIO cục bộ.

Backend yêu cầu Node.js phiên bản từ 20 đến dưới 23; cấu hình của dự án khuyến nghị Node 20 LTS. Frontend gọi API thông qua Medusa JS SDK; địa chỉ backend và publishable key được lấy từ biến môi trường.

2.3 Danh Sách Use Case

Mã	Use case	Tác nhân	Kết quả mong đợi
UC01	Duyệt catalog	Mọi khách	Xem danh sách, danh mục và chi tiết sản phẩm đang bán.
UC02	Tìm kiếm sản phẩm	Mọi khách	Nhận kết quả theo từ khóa, bộ lọc và thứ tự; fallback khi máy tìm kiếm lỗi.
UC03	Quản lý giỏ hàng	Mọi khách	Thêm, xóa, đổi số lượng; lưu cục bộ và đồng bộ phiên khi có Redis.
UC04	Đăng ký/đăng nhập	Khách hàng	Nhận JWT hợp lệ và truy cập chức năng tài khoản.
UC05	Đặt hàng	Khách hoặc khách hàng	Backend xác minh giá, tồn kho, phí ship, mã giảm giá và tạo đơn.
UC06	Xem lịch sử đơn	Khách hàng	Chỉ xem các đơn gắn với định danh khách đang đăng nhập.
UC07	Hộp tự chọn	Mọi khách	Chọn thành phần hộp theo cấu hình sản phẩm và thêm vào giỏ.
UC08	Tư vấn chatbot	Mọi khách	Trả lời dựa trên FAQ và kết quả catalog; ghi log câu hỏi chưa giải quyết.
UC09	Quản lý catalog, khuyến mãi, tồn kho	Quản lý cửa hàng	Tạo/sửa sản phẩm, danh mục, mã giảm giá, công thức và dữ liệu tồn kho.
UC10	Quản lý đơn hàng và khách hàng	Quản lý cửa hàng	Tra cứu chi tiết, cập nhật trạng thái xử lý đơn và theo dõi hồ sơ khách hàng.
UC11	Quản lý nội dung, media và FAQ	Biên tập viên	Quản lý banner, blog, media, FAQ và các trang chính sách.
UC12	Báo cáo vận hành	Quản lý cửa hàng	Xem chỉ số doanh thu, chi phí, lợi nhuận và dữ liệu tổng hợp.
UC13	Quản lý người dùng và RBAC	Super Admin	Gán vai trò, quyền và giới hạn chức năng theo mô-đun/hành động.
UC14	Cấu hình giao hàng và website	Super Admin hoặc tài khoản được cấp quyền settings.*	Cập nhật phí vận chuyển, ngưỡng miễn phí, khu vực phục vụ và cấu hình website.

Hình 2.1 nhóm các use case theo hai vùng khách hàng và quản trị. Quan hệ kế thừa actor được dùng để tránh lặp đường nối: khách hàng kế thừa khả năng của mọi khách; Super Admin kế thừa phạm vi thao tác của biên tập viên và quản lý cửa hàng, đồng thời có thêm quyền RBAC và cấu hình hệ thống. Đây là sơ đồ tổng quan; các điều kiện và luồng thay thế được đặc tả chi tiết ở các mục tiếp theo.



Hình 2.1. Use case tổng quan của hệ thống Mộng Fruitboxz

2.3.1 Quy ước mô hình hóa

Các sơ đồ trong báo cáo tuân theo quy ước UML được sử dụng trong học phần: tác nhân nằm ngoài biên hệ thống; use case được biểu diễn bằng hình ellipse; quan hệ *include/extend* dùng đường nét đứt và mũi tên phụ thuộc; quan hệ kế thừa actor dùng nét đứt với tam giác rỗng hướng về actor tổng quát hơn; lớp có các ngăn tên–thuộc tính; biểu đồ trình tự đọc từ trên xuống; trạng thái chỉ nối bằng các chuyển tiếp có ý nghĩa nghiệp vụ. Màu sắc chỉ dùng để phân vùng hoặc làm tiêu đề, không mã hóa thêm ngữ nghĩa khi chưa có chú giải. Phần trình bày được tách rõ giữa sơ đồ phân tích, ảnh giao diện và sơ đồ triển khai.

2.4 Đặc Tả Các Luồng Nghiệp Vụ Chính

2.4.1 UC02 – Tìm kiếm sản phẩm

Thuộc tính	Đặc tả
Tác nhân	Mọi khách truy cập.
Tiền điều kiện	Frontend kết nối được backend; Meilisearch là thành phần tối ưu tùy chọn.
Kích hoạt	Người dùng nhập từ khóa hoặc đổi danh mục, mức giá, thứ tự sắp xếp.
Luồng chính	(1) Gửi tiêu chí tìm kiếm; (2) chỉ lấy product published; (3) áp dụng filter, sort và phân trang; (4) trả tên, slug, ảnh, giá, danh mục và trạng thái tồn.
Luồng thay thế	Nếu Meilisearch lỗi hoặc chưa cấu hình, backend dùng Medusa Query, chuẩn hóa chuỗi và lọc catalog tại ứng dụng.
Hậu điều kiện	Danh sách kết quả và tổng số bản ghi khớp tiêu chí được hiển thị; dữ liệu không bị thay đổi.

2.4.2 UC03 – Quản lý giỏ hàng

Thuộc tính	Đặc tả
Tác nhân	Mọi khách truy cập.
Tiền điều kiện	Sản phẩm có biến thể hợp lệ và còn khả năng mua.
Luồng chính	Thêm sản phẩm, đổi số lượng, chọn dòng hàng hoặc xóa; reducer cập nhật trạng thái và lưu localStorage.
Đồng bộ	Mỗi trình duyệt có session ID; frontend thử đồng bộ qua /store/session-cart/:id khi Redis hoạt động.
Luồng suy giảm	Redis lỗi không chặn mua hàng; localStorage trở thành nguồn dữ liệu hoạt động. Ảnh cũ được đổi chiều lại thumbnail từ catalog.
Hậu điều kiện	Tổng số lượng và tạm tính được cập nhật; giỏ còn tồn tại sau khi tải lại trang.

2.4.3 UC05 – Đặt hàng

Thuộc tính	Đặc tả
Tác nhân	Khách vắng lai hoặc khách hàng đã đăng nhập.
Tiền điều kiện	Giỏ có ít nhất một dòng hàng; tên, điện thoại và địa chỉ hợp lệ. Email là tùy chọn.
Đầu vào	variant_id, số lượng, thông tin người nhận, địa chỉ hoặc tọa độ, mã khuyến mãi; backend hỗ trợ khóa idempotency tùy chọn.

Luồng chuẩn bị	Người dùng có thể cho phép định vị. Frontend gọi reverse geocode để điền địa chỉ; mỗi khi địa chỉ/tọa độ thay đổi, hiệu ứng debounce 300 ms tự gọi <code>/store/shipping/quote</code> và cập nhật phí, chế độ tính cùng khoảng cách ước lượng.
Luồng chính	(1) Xác thực payload; (2) nếu có khóa idempotency thì tìm đơn đã xử lý trong phạm vi khách đang đăng nhập hoặc email khách vắng lai; (3) workflow tải lại biến thể và giá VND; (4) tính nhu cầu BOM, kiểm tra tồn; (5) tính lại ship và promotion tại server; (6) tạo order pending và mã MONG-...; (7) phát order <code>.placed</code> ; (8) trả order và bảng tổng hợp.
Luồng lỗi	Từ chối khi biến thể không tồn tại, thiếu giá, thiếu nguyên liệu, promotion không hợp lệ hoặc dữ liệu giao hàng sai. Nếu reverse geocode lỗi, giữ tọa độ và yêu cầu người dùng bổ sung địa chỉ; nếu API quote lỗi, frontend dùng ước lượng cục bộ.
Quy tắc tin cậy	Không dùng giá/tổng tiền gửi từ trình duyệt làm nguồn chuẩn. Phí giao hàng và giảm giá được tính lại ở backend; idempotency chỉ có hiệu lực khi client cung cấp khóa và khóa được kiểm tra theo đúng người dùng/email tương ứng. Giao diện checkout hiện chưa sinh khóa này, nhưng API đã có điểm mở rộng để bổ sung.
Hậu điều kiện	Order và metadata giao hàng được lưu; frontend xóa giỏ và hiển thị mã đơn, tổng thanh toán cùng QR chuyển khoản.

Hình 3.3 mô tả luồng nghiệp vụ từ góc nhìn trách nhiệm; Hình 3.4 mô tả bước định vị và báo phí; Hình 3.5 mô tả các bước kiểm tra khi tạo đơn. Vì vậy, UC05 không chỉ được mô tả bằng bảng mà còn có sơ đồ thể hiện thứ tự xử lý và trách nhiệm giữa frontend, backend và các mô-đun liên quan.

Bảng quyết định phí giao hàng

Điều kiện	Chế độ	Kết quả
Chưa có địa chỉ/tọa độ	<code>fallback-empty</code>	Dùng phí Hà Nội mặc định để tạm tính.
Ngoài Hà Nội	<code>static-non-hanoi</code>	Dùng mức phí ngoài Hà Nội từ cấu hình.
Quận thuộc danh sách miễn phí	<code>district-free</code>	Phí giao hàng bằng 0.

Khớp quận hoặc có tọa độ	distance-estimate	Khoảng cách đường đi ước lượng bằng Haversine $\times 1,3$; phí = phí nền + đơn giá/km, làm tròn nghìn và chặn min-max.
Có địa chỉ Hà Nội nhưng không khớp quận	static-hanoi	Dùng phí Hà Nội mặc định.
API quote không khả dụng	fallback ở frontend	Ước lượng cục bộ theo quận đã nhận diện; nếu không nhận diện được thì dùng phí mặc định.

2.4.4 UC08 – Chatbot tư vấn

Thuộc tính	Đặc tả
Tác nhân	Mọi khách truy cập.
Đầu vào	Câu hỏi tự nhiên; không nhận dữ liệu đơn hàng hoặc thông tin thanh toán.
Luồng chính	Chuẩn hóa câu hỏi, tìm FAQ và tối đa bốn sản phẩm liên quan, tạo prompt có ngữ cảnh rồi gọi Groq.
Ràng buộc	Không bịa sản phẩm, không tiết lộ prompt, không truy cập dữ liệu cá nhân; có rate limit theo IP và cache câu trả lời.
Luồng thay thế	Trả FAQ trực tiếp; nếu AI lỗi thì dùng kết quả catalog hoặc hướng dẫn liên hệ hotline.
Hậu điều kiện	Nội dung trả lời, chế độ phản hồi và trạng thái resolved được ghi vào chatbot log.

2.4.5 UC13 – Phân quyền quản trị

Thuộc tính	Đặc tả
Tác nhân	Người dùng quản trị đã được cấp tài khoản.
Tiền điều kiện	JWT admin hợp lệ; metadata chứa danh sách role hiện hành.
Luồng chính	Tải permission; giao diện dùng RequirePermission; backend ánh xạ route và HTTP method sang module.action, sau đó cho phép hoặc trả 403.
Quy tắc	Kiểm tra quyền bắt buộc ở backend; ẩn menu chỉ là biện pháp UX. Super admin dùng permission đại diện *.

Hậu điều kiện	Chỉ các route và thao tác được cấp quyền mới thực thi; thay đổi role phản ánh khi tải lại quyền.
---------------	--

2.4.6 UC14 – Cấu hình giao hàng và website

Thuộc tính	Đặc tả
Tác nhân	Super Admin hoặc tài khoản quản trị được cấp quyền <code>settings.*</code> .
Tiền điều kiện	JWT admin hợp lệ; người dùng có quyền cấu hình tương ứng, đặc biệt là nhóm quyền <code>settings</code> .
Luồng chính	Cập nhật phí giao hàng mặc định, ngưỡng miễn phí, danh sách khu vực miễn phí, tọa độ cửa hàng, nội dung website và các trang chính sách.
Ràng buộc	Các giá trị ảnh hưởng trực tiếp tới checkout như phí giao hàng và tọa độ xuất phát phải được lưu tại backend; frontend chỉ hiển thị và gửi yêu cầu cập nhật.
Hậu điều kiện	API báo phí và luồng checkout sử dụng cấu hình mới trong các lần tính tiếp theo.

2.5 Truy Vết Yêu Cầu Sang Thiết Kế

Use case	Mô hình hành vi	Thành phần thực thi	Bằng chứng giao diện
UC02	Use case tổng quan	Search API, Meilisearch, fallback Query	Hình 5.1
UC03	Activity checkout, bước chuẩn bị giỏ	CartContext, session cart	Hình 5.3
UC05	Activity, luồng định vị/báo phí, luồng tạo đơn, state order	Checkout UI/API, Shipping module, Ingredient, Order, Event Bus	Hình 5.3
UC08	Sequence chatbot	Chatbot API, FAQ/catalog retrieval, Groq	Hình 5.3
UC09–UC14	Module, class, ERD và deployment	Admin routes, RBAC, module service	Hình 5.4, 5.5 và 5.6

2.6 Yêu Cầu Phi Chức Năng

Thuộc tính	Yêu cầu và cách kiểm chứng
Hiệu năng	Route giao diện được tách bundle; ảnh tải lười; tìm kiếm ưu tiên chỉ mục Meilisearch và có phân trang.
Tin cậy	Checkout hỗ trợ khóa idempotency ở backend; giá được xác minh tại server; cache, Redis và tìm kiếm đều có phương án suy giảm chức năng.
Bảo mật	JWT tách customer/admin; backend kiểm tra RBAC; payload được xác thực; secret lấy từ biến môi trường.
Khả dụng	Luồng browse–cart–checkout có thông báo loading/error; các trang chính hoạt động ở kích thước desktop và mobile.
Bảo trì	API chia namespace, nghiệp vụ chia module, migration đi cùng module và thành phần giao diện được tái sử dụng.
Quan sát	Lỗi API có mã trạng thái rõ; chatbot lưu log câu hỏi; production cần bổ sung structured log và metric.

2.7 Yêu Cầu Dữ Liệu và Kiểm Tra Hợp Lệ

- Định danh sản phẩm và biến thể phải tồn tại tại thời điểm checkout.
- Số lượng mua phải là số nguyên dương và không vượt quá lượng nguyên liệu khả dụng.
- Giá đơn hàng luôn được lấy từ backend và dùng tiền tệ VND.
- Khóa idempotency của checkout chỉ được đối chiếu trong phạm vi khách đang đăng nhập hoặc email khách vắng lại.
- Email, số điện thoại, địa chỉ và payload contact/chatbot được kiểm tra qua middleware Zod.
- Khách vắng lại không được tự động gắn đơn vào tài khoản đã tồn tại chỉ dựa trên email.
- API admin yêu cầu xác thực; các mô-đun nhạy cảm yêu cầu thêm quyền RBAC tương ứng.

2.8 Tiêu Chí Chấp Nhận

Nhóm	Tiêu chí
Catalog	Sản phẩm xuất bản hiển thị đúng tên, ảnh, giá, biến thể và danh mục.

Nhóm	Tiêu chí
Checkout	Không tạo đơn nếu giá/tồn kho không hợp lệ; tổng tiền thể hiện được giảm giá và phí giao hàng.
Bảo mật	Route admin từ chối người chưa xác thực; người thiếu quyền nhận mã 403.
Khả dụng	Giỏ hàng vẫn hoạt động khi Redis lỗi; tìm kiếm có kết quả fallback khi Meilisearch lỗi.
Quản trị	Thay đổi catalog/nội dung được phản ánh ở frontend và chỉ mục liên quan được vô hiệu hóa hoặc cập nhật.
Responsive	Các luồng mua hàng chính sử dụng được trên màn hình desktop và thiết bị di động.

2.9 Giới Hạn Phạm Vi Hiện Tại

Phiên bản được rà soát chưa tích hợp cổng thanh toán trực tuyến; đơn mới được ghi nhận với trạng thái `not_paid`. Subscriber email mới ghi nội dung ra log thay vì gửi qua nhà cung cấp thực. Các phần sau bán hàng như gửi thông báo, đối soát thanh toán và chăm sóc đơn vẫn còn ở mức phác thảo, chưa nên xem là chức năng hoàn chỉnh.

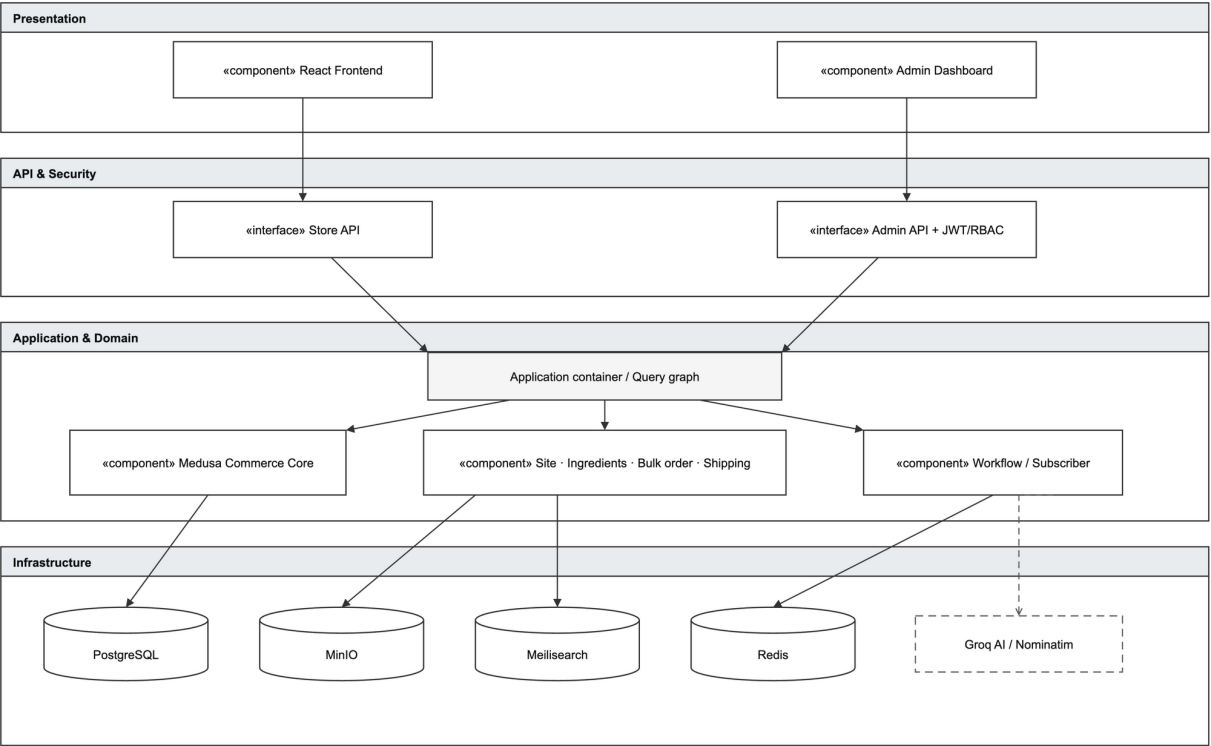
Chương 3: Thiết Kế Hệ Thống

3.1 Chuyển Từ Phân Tích Sang Thiết Kế

Từ các use case ở Chương 2, hệ thống được chia thành ba nhóm trách nhiệm: giao tiếp người dùng, điều phối nghiệp vụ và hạ tầng dữ liệu. Các danh từ miền nghiệp vụ như sản phẩm, biến thể, nguyên liệu, đơn hàng và quyền được chuyển thành lớp hoặc module; các động từ như tìm kiếm, đặt hàng, kiểm tra tồn và duyệt nội dung được chuyển thành API, workflow hoặc subscriber. Biểu đồ lớp miền mô tả cấu trúc ổn định, biểu đồ hoạt động mô tả trách nhiệm giữa các vùng xử lý, còn biểu đồ trình tự làm rõ thứ tự gọi tại runtime.

3.2 Kiến Trúc Tổng Thể

Hệ thống áp dụng kiến trúc headless commerce. Giao diện React không truy cập cơ sở dữ liệu trực tiếp mà giao tiếp với Medusa qua JS SDK và các API tùy biến. Các dịch vụ dữ liệu được triển khai độc lập, nhờ đó có thể thay đổi giao diện hoặc mở rộng hạ tầng mà không làm thay đổi toàn bộ lõi nghiệp vụ.



Hình 3.1. Kiến trúc logic của Mộng Fruitboxz

3.3 Thiết Kế Frontend

3.3.1 Tổ chức ứng dụng

Frontend được xây dựng dưới dạng single-page application. React Router ánh xạ URL tới trang; React.lazy và Suspense tách bundle theo route. Các provider ở cấp ứng dụng quản lý catalog, giỏ hàng, ngôn ngữ, xác thực khách hàng và xác thực quản trị.

Thành phần	Trách nhiệm
apiFetch	Bọc <code>medusa.client.fetch</code> , tự xử lý timeout, token và cấu trúc response envelope.
CatalogContext	Cung cấp dữ liệu sản phẩm và danh mục cho các trang mua sắm.
CartContext	Quản lý reducer giỏ hàng, localStorage, đồng bộ Redis và thông báo thêm hàng.
AuthContext	Đọc JWT khách hàng, kiểm tra hết hạn và cung cấp trạng thái đăng nhập.
AdminAuthContext	Đăng nhập admin, tải quyền, xử lý 401 và kiểm tra quyền theo chức năng.
LanguageContext	Kết hợp i18next để hỗ trợ nội dung tiếng Việt và tiếng Anh.

3.3.2 Nguyên tắc giao tiếp API

Medusa SDK được cấu hình với backend URL, publishable key và cơ chế lưu JWT. API lỗi và API tùy biến đều đi qua một lớp gọi chung để nhất quán timeout và xử lý lỗi. Riêng tìm kiếm có thể gọi Meilisearch trực tiếp khi cấu hình search key phía client; nếu không, giao diện gọi API backend.

3.4 Thiết Kế Backend Medusa

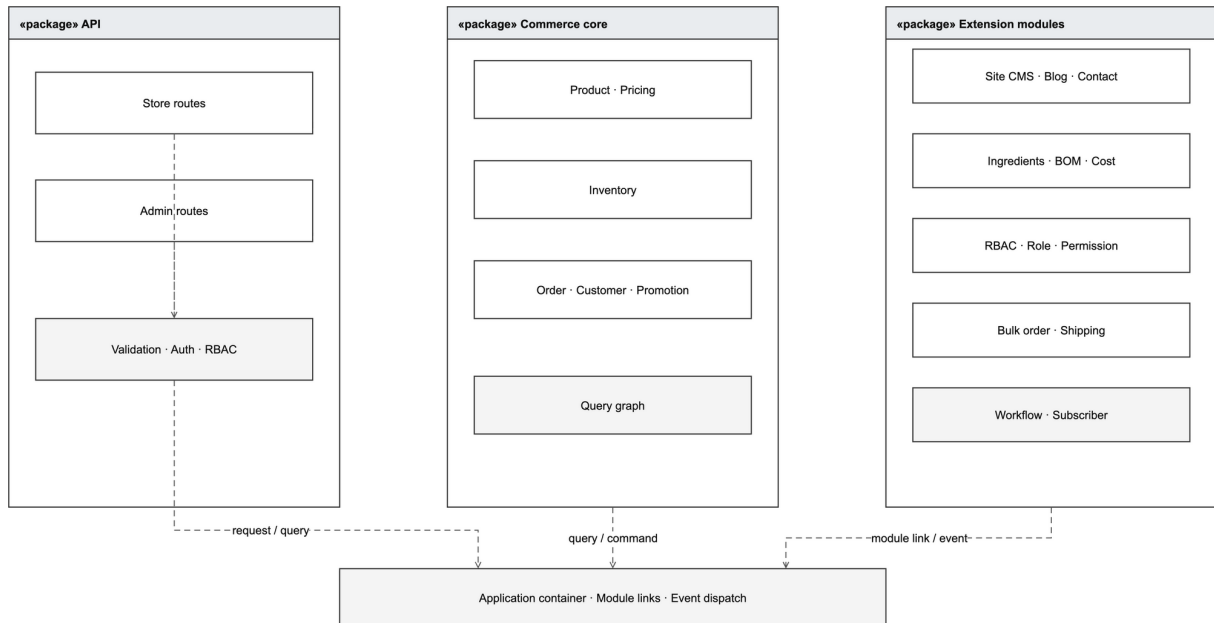
3.4.1 Mô-đun lõi và mô-đun mở rộng

Medusa cung cấp các mô-đun lõi như Product, Pricing, Inventory, Order, Customer, User, Promotion và Region. Báo cáo tập trung vào năm mô-đun mở rộng đang tham gia trực tiếp vào luồng chức năng hiện hành:

- **Site:** banner, cấu hình website, blog, liên hệ và log chatbot.
- **RBAC:** role, permission và ánh xạ quyền của người dùng quản trị.
- **Ingredients:** nguyên liệu, công thức BOM và kiểm tra/trừ tồn kho.
- **Bulk orders:** lưu yêu cầu đặt hàng số lượng lớn.

- **Shipping:** gợi ý khu vực, reverse geocode và tính phí giao hàng theo cấu hình cửa hàng.

Các module link nối User với Role và Ingredient với InventoryItem mà không đưa khóa ngoại của mô-đun khác trực tiếp vào model tùy biến.



Hình 3.2. Cấu trúc mô-đun nghiệp vụ và điểm tích hợp

3.4.2 API, workflow và subscriber

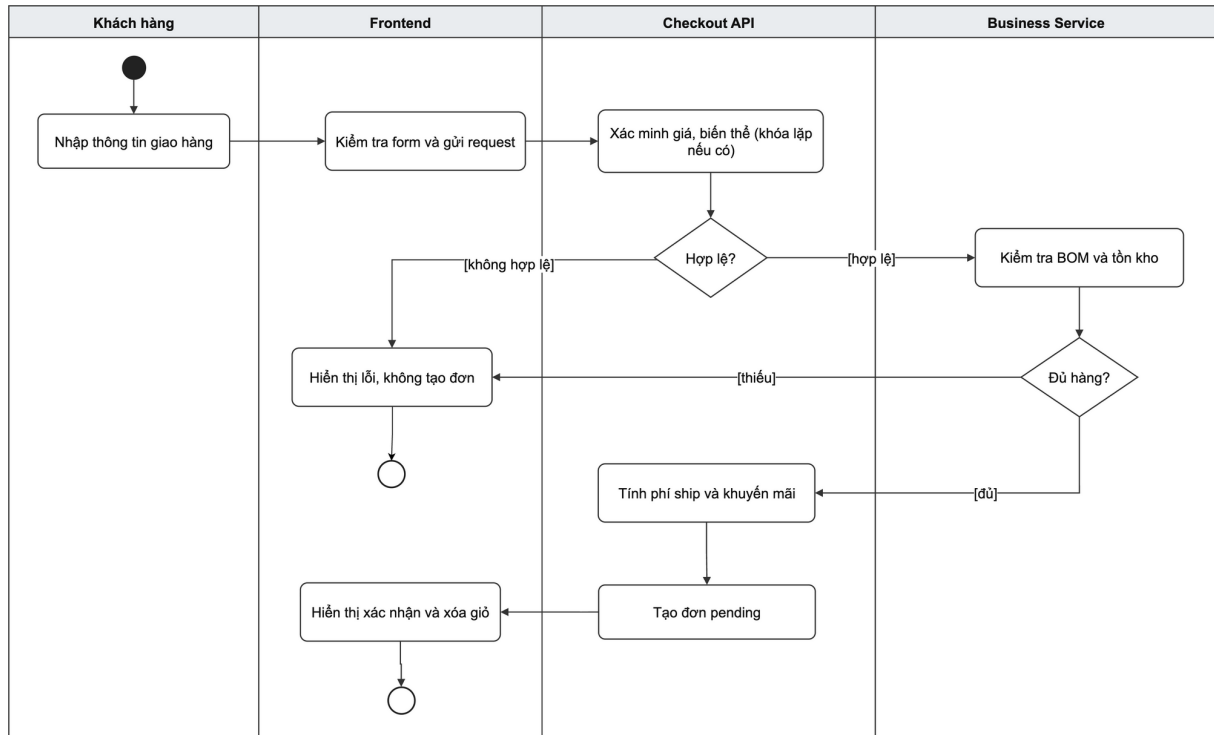
API được chia theo hai namespace: `/store` dành cho khách hàng và `/admin` dành cho vận hành. Middleware chung thực hiện xác thực, RBAC, validation và chuẩn hóa response. Workflow được dùng cho luồng tạo contact message và checkout; riêng checkout đã tách các bước xác minh giá, kiểm tra tồn kho, tính phí giao hàng, áp dụng khuyến mãi, tạo đơn, cập nhật mã đơn và phát sự kiện. Các step ghi dữ liệu có compensation: tạo đơn có thể xóa lại order nếu workflow lỗi, cập nhật mã đơn có thể khôi phục metadata cũ; riêng sự kiện `order.placed` là tác vụ hậu xử lý nên được đặt sau khi đơn đã tạo. Các thay đổi sản phẩm và đơn hàng được xử lý tiếp bằng subscriber:

- Sản phẩm tạo/cập nhật/xóa sẽ đồng bộ tài liệu tương ứng trong Meilisearch và vô hiệu cache.
- Danh mục thay đổi sẽ xóa cache danh mục và cache tìm kiếm.
- Đơn được tạo sẽ phát sự kiện `order.placed`; subscriber trừ tồn nguyên liệu và chuẩn bị email xác nhận.

3.5 Luồng Checkout

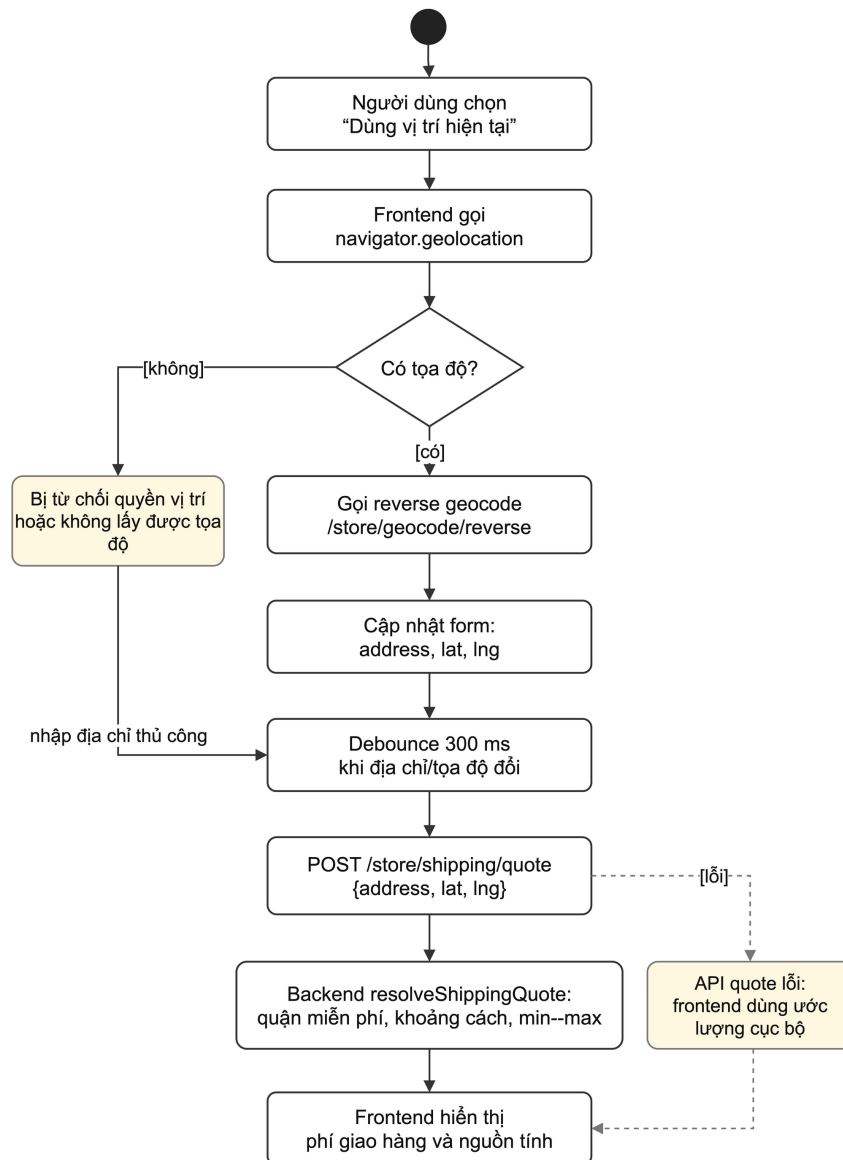
3.5.1 Biểu đồ hoạt động

Biểu đồ hoạt động trong Hình 3.3 tách trách nhiệm thành bốn swimlane. Frontend chỉ kiểm tra dữ liệu ở mức giao diện; Checkout API xác minh lại dữ liệu tin cậy; dịch vụ nghiệp vụ chịu trách nhiệm tồn kho, phí giao hàng và khuyến mãi. Hai nút quyết định thể hiện rõ đường thành công và đường quay lại sửa dữ liệu.



Hình 3.3. Biểu đồ hoạt động của quy trình checkout

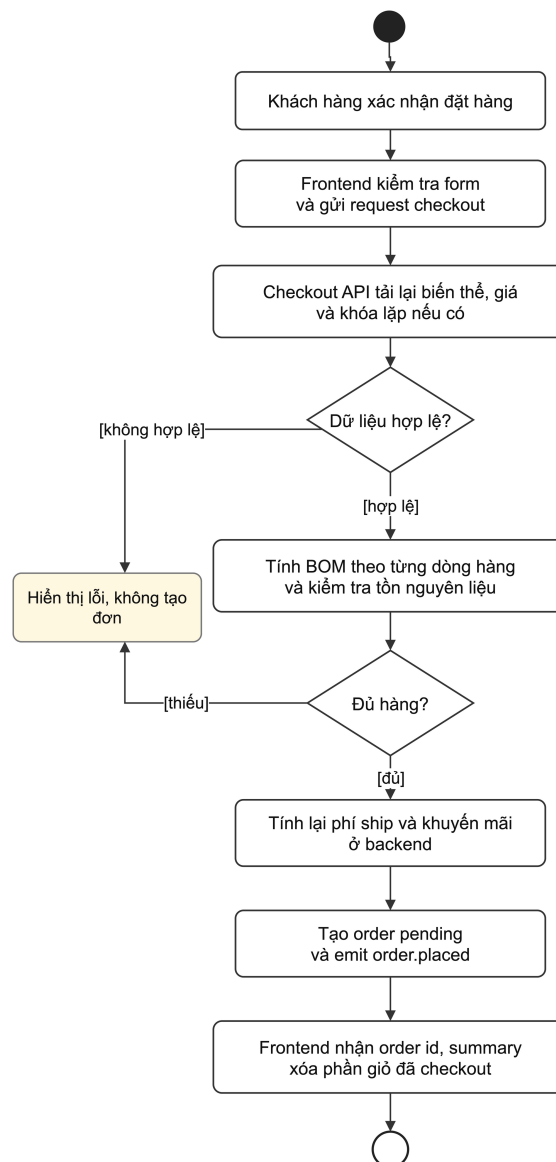
3.5.2 Định vị và tự động báo phí giao hàng



Hình 3.4. Sơ đồ luồng định vị và tự động tính phí giao hàng

Khi người dùng chọn vị trí hiện tại, trình duyệt trả tọa độ cho frontend. Frontend gọi reverse geocode để điền địa chỉ có thể đọc được; thay đổi của địa chỉ hoặc tọa độ kích hoạt yêu cầu báo phí sau 300 ms. API sử dụng `ShippingModuleService` để đọc cấu hình điểm xuất phát, danh sách quận miễn phí và công thức khoảng cách rồi trả phí cùng mode. Checkout workflow vẫn tính lại cùng quy tắc khi tạo đơn, vì số tiền hiển thị phía client không phải nguồn dữ liệu tin cậy.

3.5.3 Sơ đồ luồng xử lý đặt hàng

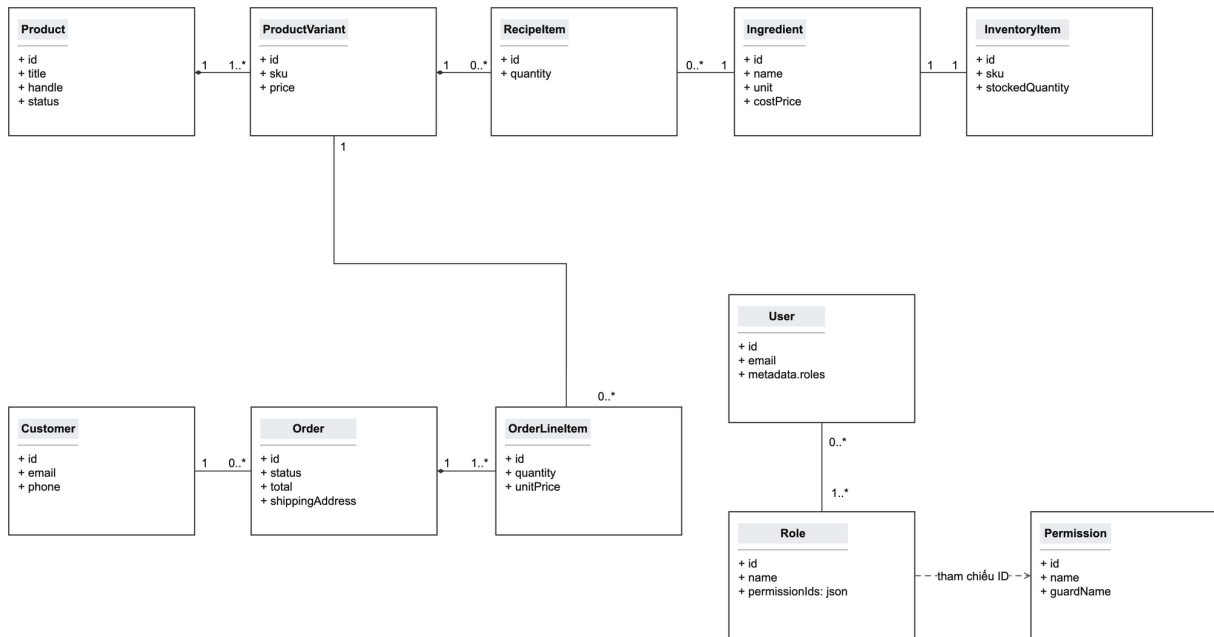


Hình 3.5. Sơ đồ luồng xử lý đặt hàng ở checkout

Điểm quan trọng của thiết kế là backend không chấp nhận giá do trình duyệt gửi. Route checkout chỉ chuẩn hóa request, kiểm tra khóa idempotency theo phạm vi khách hàng/email và gọi `checkoutOrderWorkflow`. Workflow lần lượt thực hiện `resolvePricesStep`, `validateInventoryStep`, `resolveShippingStep`, `resolvePromotionsStep`, tạo đơn, cập nhật mã đơn và phát sự kiện `order.placed`. Cách tách bước này giúp luồng đặt hàng rõ trách nhiệm hơn, có điểm hoàn tác rõ cho các thao tác ghi dữ liệu, đồng thời thuận lợi khi cần bổ sung thanh toán trực tuyến ở giai đoạn sau.

3.6 Mô Hình Lớp Miền

Hình 3.6 giữ lại các lớp có ảnh hưởng trực tiếp đến catalog, công thức nguyên liệu, đơn hàng và RBAC. Product sở hữu nhiều ProductVariant; mỗi biến thể có nhiều RecipeItem nối tới Ingredient; Ingredient được ánh xạ tới một InventoryItem để theo dõi lượng tồn. Order sở hữu các dòng OrderLineItem và mỗi dòng tham chiếu đúng một biến thể tại thời điểm mua.



Hình 3.6. Biểu đồ lớp miền nghiệp vụ chính

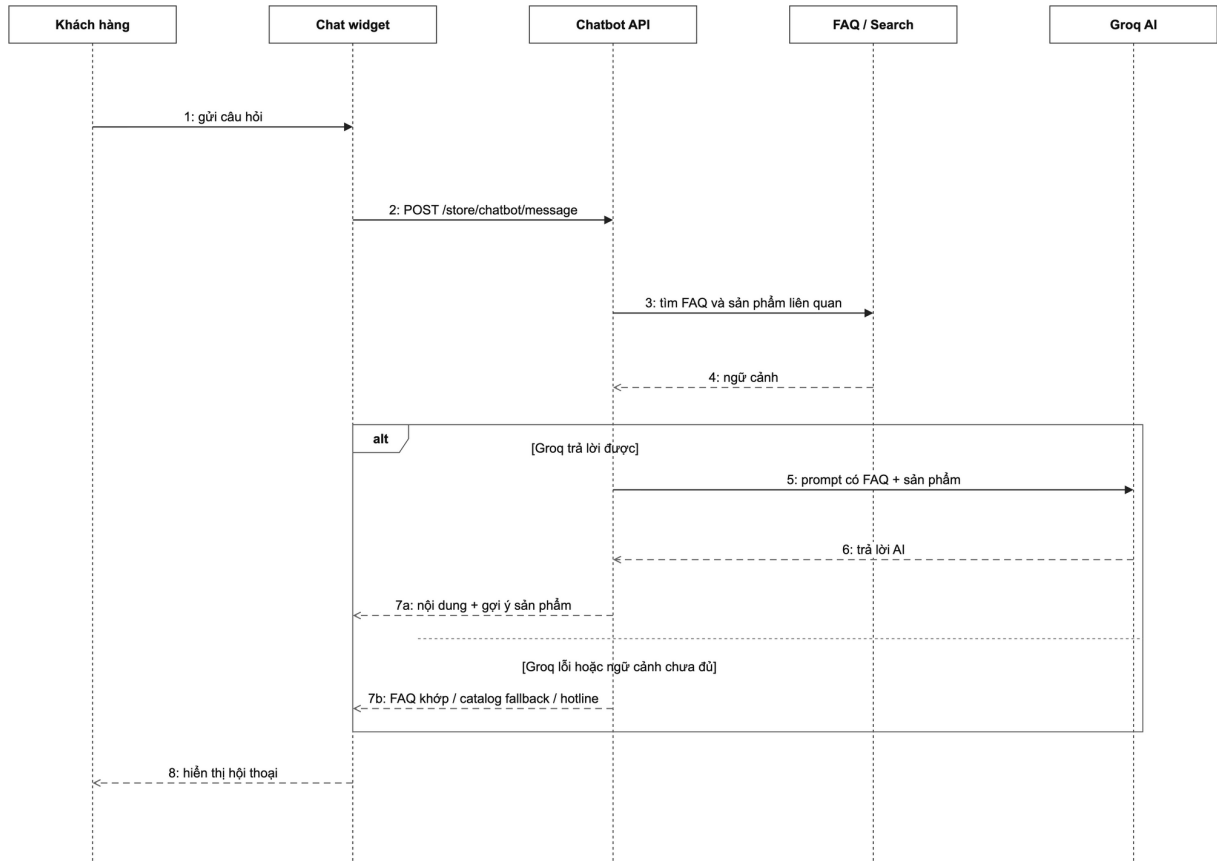
Các bộ số trên quan hệ giúp kiểm tra mô hình trước khi cài đặt. Ví dụ, một Order không nên tồn tại mà không có OrderLineItem; một ProductVariant có thể chưa được cấu hình công thức; Role và Permission có quan hệ nhiều–nhiều. Các lớp trong sơ đồ là mô hình logic, không đồng nghĩa mỗi quan hệ phải được cài bằng khóa ngoại trực tiếp vì Medusa sử dụng module link để bảo toàn ranh giới mô-đun.

3.7 Tìm Kiếm và Chatbot

Tài liệu tìm kiếm chứa tên, mô tả, slug, danh mục, mức giá, giá nhỏ nhất/lớn nhất, ảnh và trạng thái tồn. Subscriber giữ chỉ mục đồng bộ với catalog. Khi tìm kiếm chính không hoạt động, backend xây tài liệu từ Query API và lọc tại ứng dụng. Phần làm giàu trạng thái tồn theo công thức nguyên liệu được tách thành hàm dùng chung, nên Search API và chatbot cùng tránh gọi ý các biến thể không còn đủ nguyên liệu.

Chatbot sử dụng mô hình retrieval-augmented generation ở mức đơn giản: truy vấn FAQ và sản phẩm liên quan, tạo system prompt có giới hạn quyền hạn, sau đó gọi Groq. FAQ không được xem là nhánh trả lời trực tiếp trong luồng chính; nếu FAQ chưa đủ thông tin thì ngữ cảnh sản

phẩm và quy tắc trong prompt buộc chatbot hỏi thêm nhu cầu hoặc hướng dẫn liên hệ hotline thay vì bịa dữ liệu. Khi Groq lỗi, backend mới dùng fallback theo thứ tự: FAQ khớp điểm cao, gợi ý catalog, hoặc câu trả lời hotline và ghi log chưa giải quyết. Response được cache 12 giờ theo câu hỏi. Log lưu message đã chuẩn hóa, chế độ phản hồi, trạng thái resolved và số gợi ý; nhờ đó quản trị viên có thể bổ sung FAQ cho câu hỏi chưa được giải quyết.



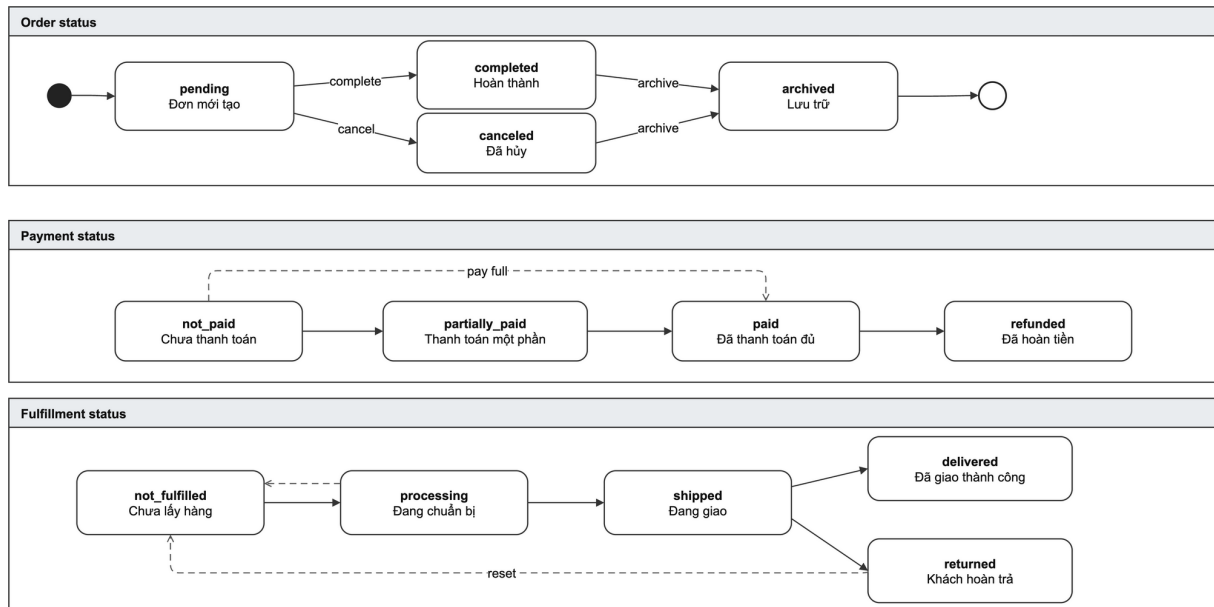
Hình 3.7. Biểu đồ trình tự của chatbot tư vấn sản phẩm

3.8 Bảo Mật

- JWT tách riêng cho khách hàng và người dùng quản trị; token hết hạn bị xóa khỏi localStorage.
- API admin sử dụng middleware xác thực và RBAC; quyền được kiểm tra lại ở backend, không chỉ dựa vào việc ẩn menu.
- Checkout khách được cho phép nhưng không tự liên kết với customer có cùng email, tránh chiếm quyền lịch sử đơn.
- Chatbot có rate limit, phạm vi dữ liệu hạn chế và quy tắc chống prompt injection cơ bản.
- CORS, JWT secret, cookie secret và khóa dịch vụ được cấu hình bằng biến môi trường.

3.9 Vòng Đời Đơn Hàng

Code quản trị hiện tách trạng thái đơn thành ba nhóm để tránh trộn lẫn nghiệp vụ: status của order, payment_status trong metadata và fulfillment_status trong metadata. Nhân viên không được chuyển trạng thái tùy ý; giao diện kiểm tra transition trước khi gọi API cập nhật. Ví dụ, order pending chỉ có thể sang completed hoặc canceled; trạng thái delivered của giao hàng không được chuyển ngược về shipped.



Hình 3.8. Biểu đồ trạng thái đơn hàng

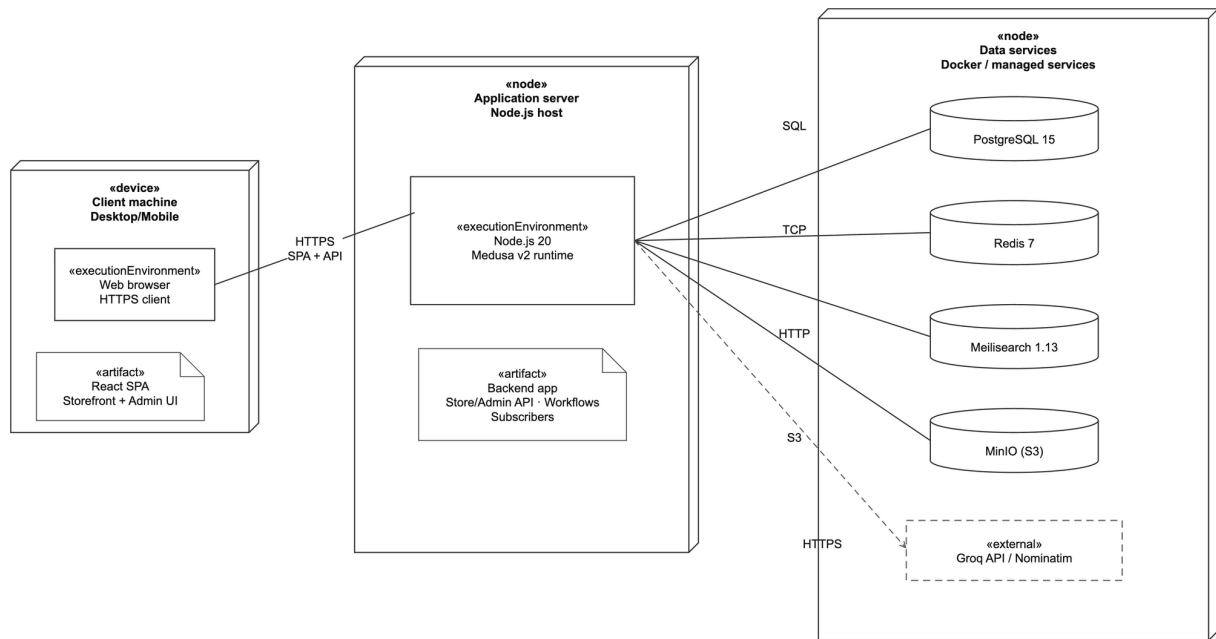
Mỗi chuyển tiếp nên được thực hiện qua một command hoặc workflow có kiểm tra trạng thái nguồn, quyền người thao tác và tác dụng phụ tương ứng. Cách này ngăn các chuyển tiếp vô nghĩa như đưa đơn đã giao thành công quay lại đang giao, đồng thời tạo điểm rõ ràng để phát sự kiện, ghi audit log và gửi thông báo.

3.10 Mô Hình Triển Khai

Trong môi trường phát triển, Docker Compose chạy bốn dịch vụ: PostgreSQL cổng 5432, Redis cổng 6379, Meilisearch cổng 7700 và MinIO (S3 API được ánh xạ ra cổng 9001, console cổng 9002). PostgreSQL dùng image pgvector và mount thư mục backend/scripts để khởi tạo schema/extension khi tạo database mới. Backend chạy ở cổng 9000; frontend Vite hiện cấu hình cổng 5173 và proxy các đường dẫn /auth, /store, /admin, /media tới backend.

Biểu đồ triển khai ở Hình 3.9 được mô hình hóa ở mức logic. Node client thể hiện trình duyệt và artifact React SPA sau khi tải xuống, bao gồm cả storefront và khu vực admin. Node application server thể hiện môi trường Node.js 20 chạy Medusa v2 cùng artifact backend gồm Store/Admin

API, workflow và subscriber. Nhóm data services có thể chạy bằng Docker Compose trong phát triển hoặc thay bằng dịch vụ quản lý ở production.



Hình 3.9. Mô hình triển khai và các dịch vụ tích hợp

Ở production, cần tách secret khỏi mã nguồn, bật HTTPS, giới hạn truy cập Meilisearch/MinIO, cấu hình Redis thật cho cache/event bus và đặt reverse proxy trước hai ứng dụng.

Chương 4: Thiết Kế Dữ Liệu

4.1 Tổng Quan

Hệ thống sử dụng PostgreSQL 15. Phần lớn bảng thương mại điện tử được Medusa quản lý thông qua module và migration; dự án bổ sung các bảng riêng cho nội dung website, phân quyền, nguyên liệu và yêu cầu mua số lượng lớn. Cấu hình giao hàng được lưu trong nhóm thiết lập website, còn logic gợi ý địa chỉ và tính phí được tách thành module nghiệp vụ riêng. Vì Medusa khuyến khích cô lập mô-đun, quan hệ giữa mô-đun lõi và mô-đun tùy biến được khai báo bằng module link.

Ngoài cơ sở dữ liệu quan hệ, hệ thống còn dùng Redis cho dữ liệu tạm thời, Meilisearch cho chỉ mục tìm kiếm và MinIO cho tệp nhị phân. PostgreSQL được cấu hình thêm pgvector trong môi trường Docker để chuẩn bị cột embedding cho sản phẩm và log chatbot, nhưng luồng tìm kiếm hiện hành vẫn dựa trên Meilisearch và fallback Query API. Các thành phần này không thay thế PostgreSQL mà phục vụ các kiểu truy cập chuyên biệt.

4.2 Nhóm Thực Thể Lõi Của Medusa

Miền	Thực thể tiêu biểu	Ý nghĩa
Catalog	Product, ProductVariant, ProductCategory, ProductImage	Thông tin sản phẩm, biến thể, danh mục và ảnh.
Giá bán	Price, PriceSet, Region	Giá theo biến thể, tiền tệ và khu vực bán hàng.
Kho	InventoryItem, InventoryLevel, StockLocation	Số lượng hàng hoặc nguyên liệu tại từng vị trí kho.
Khách hàng	Customer, AuthIdentity	Hồ sơ khách và định danh đăng nhập.
Đơn hàng	Order, OrderItem, OrderAddress	Đơn, dòng hàng, địa chỉ và metadata vận hành.
Khuyến mãi	Promotion, ApplicationMethod, Campaign	Mã giảm giá, cách tính và giới hạn chương trình.

Miền	Thực thể tiêu biểu	Ý nghĩa
Quản trị	User, AuthIdentity	Người dùng nội bộ và thông tin xác thực.

4.3 Các Model Tùy Biến

4.3.1 Mô-đun Site

Model	Thuộc tính chính	Mục đích
site_banner	title, subtitle, image, link, order, active	Banner động trên frontend, có thứ tự và trạng thái.
site_setting	key, value (JSON)	Cấu hình tổng quát như giao hàng, chính sách, hotline và chatbot.
site_blog_category	name, slug, description	Danh mục bài viết.
site_blog_post	title, slug, excerpt, content, image, author, category, published, published_at	Nội dung blog và trạng thái xuất bản.
site_contact_message	name, email, phone, message, status	Yêu cầu liên hệ từ khách hàng.
site_chatbot_question_log	message, normalized_message, response_mode, resolved, metadata	Theo dõi chất lượng câu trả lời chatbot.

4.3.2 Mô-đun RBAC

Model	Thuộc tính chính	Mục đích
role	name, description, guard_name, is_default, permissions (JSON)	Nhóm quyền của người dùng quản trị.
permission	name, description, guard_name	Quyền nguyên tử dạng module.action.

Danh sách ID role hiện được lưu trong metadata của User và đồng thời dự án có module link User–Role. Mỗi role lưu mảng ID permission trong trường JSON. Cách này linh hoạt khi seed dữ liệu nhưng cần chuẩn hóa để tránh hai nguồn liên kết cùng tồn tại lâu dài.

4.3.3 Mô-đun Ingredients

Model	Thuộc tính chính	Mục đích
ingredient	name, type, unit, cost_price	Nguyên liệu như trái cây, sốt, sữa chua hoặc thành phần khác.
recipe_item	variant_id, quantity, ingredient	Định mức một nguyên liệu cho một biến thể sản phẩm.

Một Ingredient liên kết một-một với InventoryItem của Medusa. Khi đặt q sản phẩm, lượng nguyên liệu i cần dùng được tính theo:

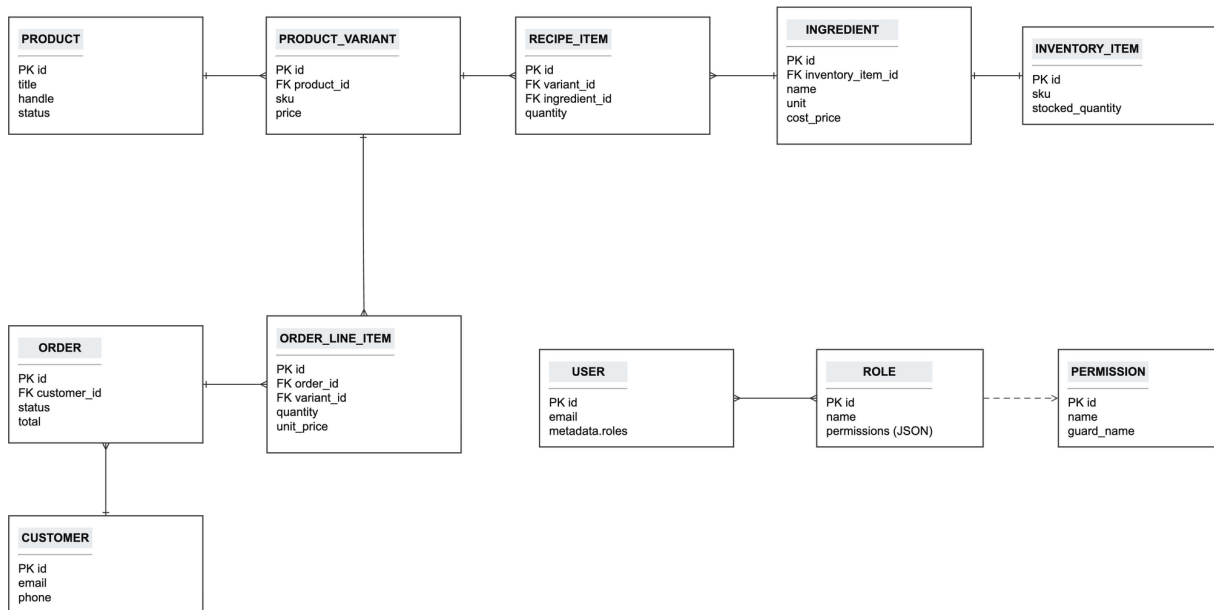
$$D_i = \sum_{v \in V} q_v \times r_{v,i} \quad (4.1)$$

trong đó q_v là số lượng biến thể v trong đơn và $r_{v,i}$ là định mức nguyên liệu i trong công thức của biến thể đó. Đơn chỉ được tạo khi tồn khả dụng của mọi nguyên liệu không nhỏ hơn D_i .

4.3.4 Các mô-đun khác

- **bulk_order**: lưu tên công ty, email, điện thoại, ngày yêu cầu, ghi chú và trạng thái của đơn số lượng lớn.
- **shipping**: không tạo bảng nghiệp vụ riêng trong phạm vi hiện tại; service đọc cấu hình từ `site_setting` để gợi ý khu vực, reverse geocode và tính phí giao hàng.

4.4 Quan Hệ Dữ Liệu



Hình 4.1. ERD logic của các thực thể nghiệp vụ chính

BlogCategory có quan hệ một-nhiều với BlogPost. User liên kết Role; Role tham chiếu Permission. Ingredient liên kết InventoryItem, còn RecipeItem nối biến thể với Ingredient.

4.5 Ánh Xạ Mô Hình Miền Sang Lưu Trữ

ERD chỉ trình bày các bảng quan trọng để giữ khả năng đọc. Trong triển khai thực tế, Medusa tách giá, kho, địa chỉ và liên kết module thành nhiều bảng chuyên biệt. Product–ProductVariant và Order–OrderLineItem là các quan hệ sở hữu; RecipeItem là thực thể kết hợp mang thuộc tính quantity; RBAC đi qua module link hoặc bảng liên kết thay vì làm rõ rí model của module lỗi vào module tùy biến.

Các khóa nghiệp vụ cần ràng buộc duy nhất gồm handle sản phẩm, SKU biến thể, email đăng nhập và tên quyền trong cùng guard. Khóa ngoại hoặc module link thường dùng để lọc cần có index, nhất là các liên kết từ order tới customer và từ recipe item tới variant/ingredient. Truy vấn danh sách cần index kết hợp theo trạng thái và thời gian tạo thay vì tải toàn bộ bản ghi rồi lọc tại ứng dụng.

Lược đồ hiện tại tập trung vào catalog, order, RBAC, site content, ingredient và inventory. Các bảng không còn phục vụ chức năng hiện hành không được đưa vào ERD để sơ đồ gọn và đúng với mã nguồn mới nhất.

4.6 Nhất Quán Giao Dịch

Checkout chạm tới nhiều loại dữ liệu: giá và catalog, số lượng nguyên liệu, promotion, order và event hậu xử lý. Luồng này hiện được đưa vào Medusa workflow để tách rõ các bước xác minh giá, tồn kho, giao hàng, khuyến mãi và tạo đơn. Workflow cung cấp transaction/execution boundary và compensation cho các step ghi dữ liệu, trong khi PostgreSQL vẫn bao phủ phần ghi giao dịch do module lõi thực hiện. Các tác vụ ngoài giao dịch như cập nhật Meilisearch hoặc gửi email nên đi qua event/outbox và có retry. Khóa idempotency được lưu trong metadata đơn và chỉ đối chiếu trong phạm vi khách hàng/email tương ứng để cùng một request lặp lại không tạo thêm đơn ngoài ý muốn. Với tồn kho cạnh tranh, bước giữ hoặc trừ hàng cần khóa/điều kiện cập nhật nguyên tử, không chỉ đọc số lượng rồi ghi lại.

4.7 Dữ Liệu Phi Quan Hệ

4.7.1 Redis

Redis lưu cache có TTL, rate-limit counter và giỏ hàng phiên. Trong development, Redis có thể tắt bằng cấu hình; hệ thống sử dụng implementation giả hoặc localStorage để giảm phụ thuộc. Trong production, Redis cũng có thể làm event bus nhằm xử lý subscriber ngoài tiến trình request.

4.7.2 Meilisearch

Mỗi tài liệu tìm kiếm được suy ra từ Product và các quan hệ liên quan. Các trường được tìm kiếm gồm title, description, slug và tên danh mục; trường lọc gồm trạng thái, category và price range; trường sort gồm thời gian và giá. PostgreSQL vẫn là nguồn dữ liệu chuẩn, còn Meilisearch là bản sao tối ưu truy vấn.

4.7.3 pgvector

Script khởi tạo `init_vector.sql` tạo extension vector, bổ sung cột embedding cho bảng product và site_chatbot_question_log, đồng thời tạo HNSW index theo cosine similarity. Phần này là nền tảng dữ liệu cho gợi ý hoặc truy xuất ngữ nghĩa trong tương lai; báo cáo không xem đây là chức năng tìm kiếm chính đã hoàn thiện.

4.7.4 MinIO

MinIO cung cấp API tương thích S3. Cơ sở dữ liệu chỉ lưu object key hoặc URL; dữ liệu nhị phân được lưu trong bucket mong-media. API media quản trị chịu trách nhiệm upload, liệt kê và xóa object.

4.8 Toàn Vẹn, Sao Lưu và Bảo Mật Dữ Liệu

- Migration của từng mô-đun phải được áp dụng đồng bộ với phiên bản mã nguồn.
- PostgreSQL và bucket MinIO cần được sao lưu; chỉ mục Meilisearch có thể dựng lại từ

catalog.

- Secret kết nối, khóa Groq, Meiliseach master key và thông tin MinIO không được dùng giá trị mặc định ở production.
- API không trả dữ liệu cá nhân cho chatbot; lịch sử đơn được lọc theo customer ID từ token.
- Checkout nhiều bước đã được tách thành workflow; nếu bổ sung thanh toán trực tuyến, phần giao dịch và hoàn tác cần được thiết kế chặt hơn.

Chương 5: Thiết Kế Giao Diện

5.1 Định Hướng Thiết Kế

Frontend sử dụng phong cách ấm, tươi và gần gũi với ngành hàng trái cây. Màu cam được dùng cho hành động chính; ảnh sản phẩm giữ vai trò trung tâm; bố cục card và khoảng trắng giúp người dùng quét nhanh danh sách. Giao diện quản trị ưu tiên mật độ thông tin, bảng dữ liệu và bộ lọc.

Các nguyên tắc chính gồm:

- **Nhất quán:** dùng chung header, footer, card sản phẩm, trạng thái và hệ thống màu.
- **Responsive:** lưới sản phẩm, form checkout và navigation thay đổi theo chiều rộng màn hình; mobile có bottom navigation.
- **Phản hồi:** các request có loading/error/empty state; thao tác giỏ hàng có toast; nút submit bị khóa trong lúc xử lý.
- **Giảm ma sát mua hàng:** giỏ được lưu lại, checkout chấp nhận khách vắng lai và phí giao hàng hiển thị trước khi đặt đơn.
- **Phân quyền:** menu và route quản trị chỉ hiển thị khi người dùng có quyền tương ứng.

5.2 Sơ Đồ Điều Hướng Frontend

Luồng mua hàng chính được tổ chức ngắn gọn theo chuỗi: *Trang chủ/Danh mục/Tìm kiếm* → *Chi tiết sản phẩm hoặc hộp tự chọn* → *Giỏ hàng* → *Checkout* → *Xác nhận đơn*. Khách đã đăng nhập có thể đi tiếp tới lịch sử đơn; blog, liên hệ và chính sách là các nhánh nội dung độc lập từ thanh điều hướng chung. Cấu trúc này giữ hành động mua hàng trong tối đa vài bước rõ ràng và vẫn cho phép khám phá nội dung thương hiệu.

5.3 Các Màn Hình Khách Hàng

Màn hình	Đường dẫn	Chức năng
Trang chủ	/	Banner động, nhóm danh mục, sản phẩm nổi bật và nội dung thương hiệu.
Danh sách sản phẩm	/products	Lưới sản phẩm, lọc, sắp xếp và điều hướng tới chi tiết.

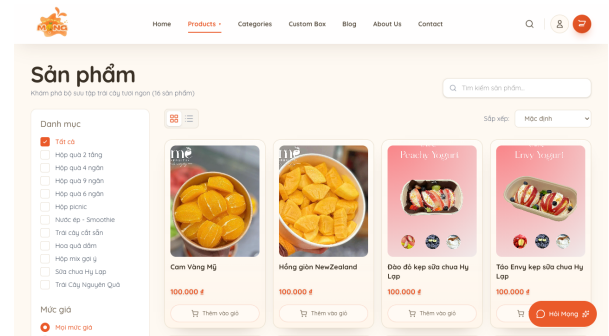
Màn hình	Đường dẫn	Chức năng
Chi tiết sản phẩm	/products/:slug	Ảnh, mô tả, biến thể, giá, số lượng, thành phần và thêm giỏ.
Danh mục	/categories/:slug	Danh sách sản phẩm trong một danh mục động.
Tìm kiếm	/search	Từ khóa, lọc danh mục/mức giá và trạng thái không có kết quả.
Hộp tự chọn	/custom-box/:slug	Tạo cấu hình hộp theo lựa chọn thành phần và ngân sách.
Giỏ hàng	/cart	Chọn dòng hàng, đổi số lượng, xóa và chuyển sang checkout.
Checkout	/checkout	Người nhận, định vị, phí ship, mã khuyến mãi, tóm tắt và đặt đơn.
Tài khoản	/account	Thông tin người dùng và điều hướng tới lịch sử đơn.
Lịch sử đơn	/order-history	Danh sách đơn của khách đang đăng nhập và mã đơn hiển thị.
Blog	/blog/:id	Danh sách và nội dung bài viết theo dữ liệu CMS.
Trang tĩnh	/about-us, /contact, các policy	Giới thiệu, liên hệ, thanh toán, giao hàng và quyền riêng tư.

5.4 Ảnh Chụp Giao Diện Từ Phiên Chạy Thử

Các hình dưới đây được chụp trực tiếp tại localhost:5173. Bộ ảnh được chọn theo hành trình sử dụng thay vì chụp mọi route: sáu màn hình khách hàng bao quát khám phá—cấu hình—tư vấn—checkout và năm màn hình quản trị bao quát KPI—catalog—đơn hàng—tồn kho—cấu hình giao hàng. Ảnh trang chủ được chụp sau khi banner động tải xong; ảnh checkout được chụp sau khi địa chỉ kích hoạt API báo phí. Dữ liệu catalog, giá, ảnh, giỏ hàng và KPI đều đến từ API của dự án, không phải mockup tĩnh.

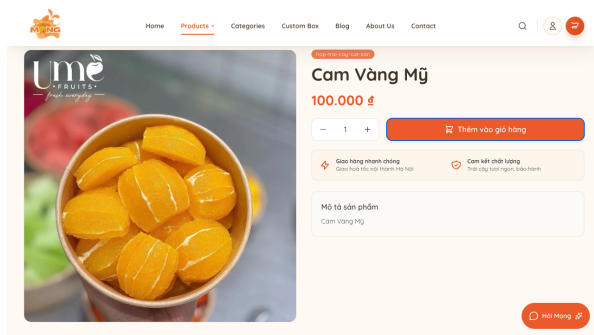


(a) Trang chủ sau khi banner động tải hoàn tất

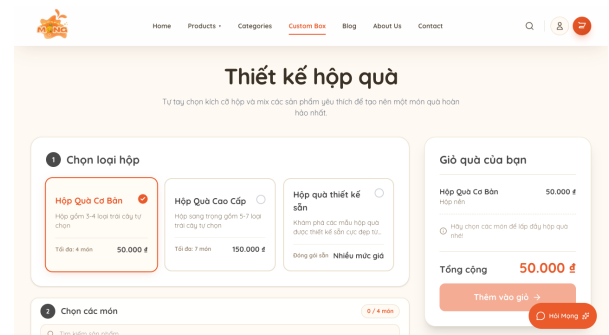


(b) Catalog và bộ lọc sản phẩm

Hình 5.1. Hai điểm vào chính của hành trình khám phá sản phẩm



(a) Chi tiết sản phẩm và biến thể

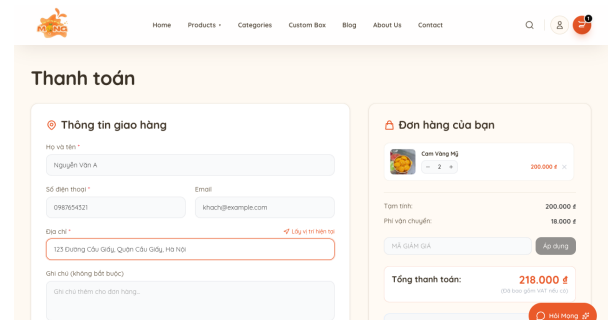


(b) Tùy chọn thành phần hộp quà

Hình 5.2. Hai cách cấu hình sản phẩm trước khi thêm vào giỏ



(a) Chatbot tư vấn trong ngữ cảnh frontend



(b) Checkout sau khi tự động cập nhật phí giao hàng

Hình 5.3. Hỗ trợ quyết định mua và hoàn tất đơn hàng

5.4.1 Trang sản phẩm và chi tiết

Card sản phẩm gom ảnh, tên, giá và hành động mua. Trang chi tiết dùng route động theo slug, nhờ đó không cần tạo file riêng cho từng sản phẩm. Khi sản phẩm có nhiều biến thể, người dùng phải chọn biến thể trước khi thêm vào giỏ. Phần dưới trang tập trung vào mô tả, thành phần nguyên liệu và sản phẩm liên quan.

5.4.2 Giỏ hàng và checkout

Giỏ hàng cho phép chọn một phần sản phẩm để checkout. Tại trang checkout, số lượng vẫn có thể chỉnh; hệ thống tính subtotal, giảm giá, phí giao hàng và tổng. Người dùng có thể lấy tọa độ hiện tại để reverse geocode hoặc nhập địa chỉ thủ công. Phí ship được debounce khi địa chỉ thay đổi và có cách tính offline dự phòng cho các quận Hà Nội.

5.4.3 Chatbot

ChatbotWidget xuất hiện trên frontend dưới dạng nút nổi. Câu trả lời hỗ trợ Markdown và có thể kèm card gợi ý sản phẩm. UI cần phân biệt trạng thái đang gửi, lỗi giới hạn tần suất và phản hồi fallback để khách hiểu mức độ chắc chắn của thông tin.

Khi chạy thử trên các trang chính, luồng khách hàng khá dễ theo dõi: nút hành động chính dùng màu cam, nội dung bán hàng dùng card và ảnh lớn, chatbot đặt ở góc nên không che checkout. Một vài điểm còn cần chỉnh là chiều cao ảnh sản phẩm, độ tương phản của chữ phụ và cách giải thích nguồn tính phí giao hàng.

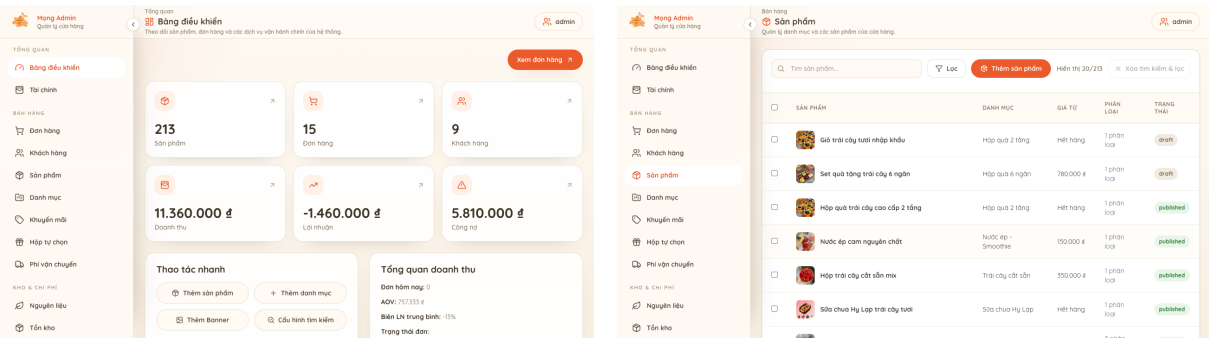
5.5 Khu Vực Quản Trị

Quản trị được đặt dưới namespace /admin. AdminLayout cung cấp sidebar, header và vùng nội dung; ProtectedRoute yêu cầu đăng nhập; RequirePermission chặn các màn hình cần quyền cụ thể.

Nhóm quản trị	Route tiêu biểu	Nhiệm vụ
Dashboard	/admin, /admin/finance	KPI tổng quan, doanh thu, chi phí và lợi nhuận.
Catalog	/admin/products, /admin/categories	CRUD sản phẩm, biến thể, ảnh và danh mục.
Kho và nguyên liệu	/admin/inventory, /admin/ingredients	Mức tồn, giá vốn, loại nguyên liệu và công thức.
Bán hàng	/admin/orders, /admin/promotions	Theo dõi đơn, trạng thái và điều kiện khuyến mãi.
Khách hàng	/admin/customers	Hồ sơ khách hàng và lịch sử mua hàng.
Nội dung	/admin/banners, /admin/blog, /admin/media	Banner, bài viết, danh mục blog và thư viện ảnh.
Cấu hình	/admin/settings/*, /admin/content/*	Giao hàng, website, chính sách và FAQ chatbot.

Nhóm quản trị	Route tiêu biểu	Nhiệm vụ
Vận hành hệ thống	/admin/search, /admin/chatbot	Trạng thái chỉ mục và câu hỏi chatbot chưa xử lý.
Phân quyền	/admin/users, /admin/roles	Người dùng nội bộ, role và permission.

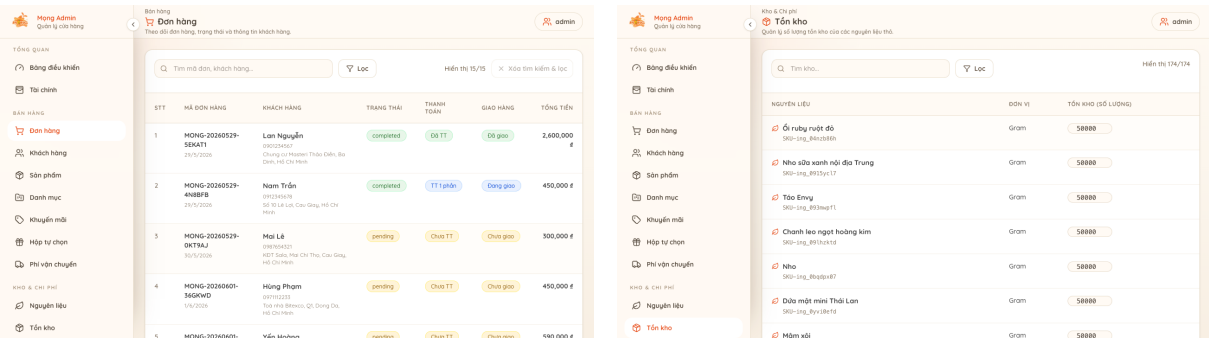
Các trang danh sách dùng chung AdminListFilters để tìm kiếm và lọc. Form tạo/sửa được tách thành route riêng. Trạng thái admin được chuẩn hóa bằng các component loading, error và empty để tránh mỗi trang tự định nghĩa cách hiển thị khác nhau.



(a) Dashboard và KPI vận hành

(b) Danh sách sản phẩm quản trị

Hình 5.4. Tổng quan vận hành và quản trị catalog



(a) Theo dõi đơn hàng và trạng thái

(b) Mức tồn kho theo mặt hàng

Hình 5.5. Hai màn hình tác nghiệp bán hàng và kho

Mong Admin
Quản lý cửa hàng

Bán hàng
Phi vận chuyển
Cấu hình các quy tắc và biểu phí giao hàng tự động.

TỔNG QUAN
Bảng điều khiển
Tài chính

BÁN HÀNG
Đơn hàng
Khách hàng
Sản phẩm
Danh mục
Khuyến mãi
Hộp tự chọn
Phi vận chuyển

KHO & CHI PHÍ
Nguyên liệu
Tồn kho

Cách hệ thống tính phí vận chuyển:

- Khách thuộc **Khu vực Freeship** hoặc đơn hàng đạt mức **Freeship** sẽ được miễn phí giao hàng (Phi = 0đ).
- Khách trong khu vực nội/ngoại thành sẽ tính phí = **Phi cơ bản + (Khoảng cách × Phi/km)** (Không vượt quá Mức phí tối đa).
- Khách ở ngoại tỉnh (không tính được khoảng cách) sẽ áp dụng **Phi ship cố định (ngoại tỉnh)**.

Chính sách Freeship & Phí mặc định

Phi ship hiển thị phụ (Fallback VND): 30000 VND

Miễn phí giao hàng (Freeship) cho đơn từ: 500000 VND

Các khu vực/Quận được Miễn phí giao hàng (cách nhau dấu phẩy):
Hoàn Kiếm, Ba Đình, Đống Đa, Hai Bà Trưng

Tọa độ cửa hàng & Tính phí theo KM

Lưu cài đặt

Lấy vị trí hiện tại

Hình 5.6. Cấu hình miễn phí giao hàng, phí mặc định và điểm xuất phát

Các bảng quản trị giữ bộ lọc và hành động ở vùng đầu trang, giúp nhân viên chuyển nhanh từ quan sát sang xử lý. Dashboard phù hợp để nhận biết xu hướng; trang đơn hàng cung cấp dữ liệu tác nghiệp; trang tồn kho hỗ trợ quyết định nhập hoặc điều chỉnh hàng. Trang cấu hình giao hàng cho phép điều chỉnh mức phí mặc định, ngưỡng miễn phí, danh sách khu vực miễn phí và tọa độ cửa hàng dùng cho báo phí. Các phần còn nên chỉnh là tên trạng thái tiếng Việt, định dạng tiền và mật độ cột giữa các bảng.

5.6 Nhận Xét Về Tính Nhất Quán

Frontend và admin hiện cùng nằm trong một ứng dụng React nhưng phục vụ hai kiểu người dùng khác nhau. Cách làm này phù hợp với phạm vi đồ án vì giúp phát triển nhanh, nhưng về lâu dài bundle và design system dễ bị phình to. Nếu tiếp tục phát triển, admin có thể được tách thành workspace riêng hoặc chuyển một số phần phù hợp sang Medusa Admin extensions; các thành phần bảng, form, màu và typography cũng nên được gom thành bộ dùng chung.

Chương 6: Tổng Kết

6.1 Kết Quả Triển Khai

Dự án đã xây dựng được một nền tảng thương mại điện tử cho Mộng Fruitboxz với hai khu vực sử dụng chính: giao diện mua sắm cho khách hàng và giao diện quản trị cho cửa hàng. Ở phía khách hàng, hệ thống hỗ trợ duyệt catalog, xem chi tiết sản phẩm, tìm kiếm, chọn biến thể, quản lý giỏ hàng, áp dụng khuyến mãi, tự động báo phí giao hàng và đặt đơn. Khách hàng đã đăng nhập có thêm khả năng quản lý tài khoản và theo dõi lịch sử mua hàng.

Ở phía vận hành, hệ thống cung cấp các chức năng quản lý sản phẩm, danh mục, đơn hàng, khách hàng, khuyến mãi, tồn kho, nguyên liệu, banner, blog, media, FAQ, cấu hình website và người dùng nội bộ. Một đặc thù nghiệp vụ được thể hiện trong đề tài là mô hình tồn kho theo công thức nguyên liệu: mỗi biến thể sản phẩm có thể được cấu thành từ nhiều nguyên liệu khác nhau, nhờ đó quá trình đặt hàng phản ánh gần hơn nghiệp vụ bán trái cây cắt sẵn và hộp quà.

Về mặt tổ chức hệ thống, dự án đã tách rõ giao diện, backend, dữ liệu và các dịch vụ hỗ trợ. Backend Medusa được mở rộng bằng các mô-đun riêng cho nội dung, phân quyền, nguyên liệu, yêu cầu mua số lượng lớn và giao hàng. Frontend sử dụng React để xây dựng trải nghiệm mua hàng và khu vực quản trị trong cùng một mã nguồn, đồng thời kết nối với các dịch vụ tìm kiếm, lưu trữ ảnh và chatbot tư vấn.

6.2 Đối Chiếu Kết Quả Theo Nhóm Chức Năng

Nhóm chức năng	Hiện trạng triển khai	Minh chứng và giới hạn
Catalog và tìm kiếm	Đã triển khai luồng chính	Sản phẩm, biến thể, danh mục, ảnh và bộ lọc đã có trên giao diện; backend có cơ chế trả kết quả dự phòng khi dịch vụ tìm kiếm không khả dụng.
Giỏ hàng và đặt hàng	Đã triển khai đặt đơn; thanh toán còn ở mức ghi nhận	Người dùng có thể thêm sản phẩm, chỉnh số lượng, nhập thông tin nhận hàng, áp dụng khuyến mãi và tạo đơn. Phí giao hàng được tính lại ở backend; thanh toán trực tuyến chưa được tích hợp.

Nhóm chức năng	Hiện trạng triển khai	Minh chứng và giới hạn
Giao hàng theo khu vực	Đã có cấu hình và báo phí tự động	Hệ thống có gợi ý địa chỉ, định vị, báo phí theo quận/khoảng cách và lưu thông tin giao hàng trong đơn; lịch giao và đối tác vận chuyển chưa được mô hình hóa sâu.
Quản trị vận hành	Đã có các màn hình nghiệp vụ chính	Khu vực quản trị bao phủ catalog, đơn hàng, khách hàng, khuyến mãi, tồn kho, nội dung và phân quyền; thao tác hàng loạt và cách gọi trạng thái vẫn cần làm chặt hơn.
Nội dung và chăm sóc khách hàng	Đã có CMS và kênh tư vấn cơ bản	Banner, blog, FAQ, liên hệ và chatbot đã được kết nối với dữ liệu dự án; chất lượng tư vấn phụ thuộc vào việc bổ sung FAQ và dữ liệu sản phẩm khi vận hành.
Tồn kho theo nguyên liệu	Đã có mô hình BOM	Công thức BOM liên kết sản phẩm bán ra với lượng nguyên liệu thực tế; nếu dùng cho cửa hàng thật, cần xử lý kỹ hơn phần cạnh tranh tồn kho và nhập/xuất kho.
Báo cáo vận hành	Có dashboard tổng quan	Dashboard đã hiển thị một số chỉ số doanh thu, chi phí, lợi nhuận và công nợ; phân tích theo thời gian, nhóm sản phẩm và hiệu quả khuyến mãi là hướng mở rộng.

6.3 Những Điểm Cần Hoàn Thiện

- **Thanh toán trực tuyến:** hiện đơn hàng chủ yếu dừng ở ghi nhận đơn và hướng dẫn chuyển khoản. Nếu triển khai thực tế, cần kết nối cổng thanh toán phù hợp và có cơ chế xác nhận trạng thái thanh toán.
- **Thông báo sau đặt hàng:** email xác nhận và thông báo cho nhân viên cửa hàng cần được kết nối với nhà cung cấp gửi thư hoặc kênh nhắn tin thật.
- **Quy trình giao vận:** trạng thái giao hàng đã được mô hình hóa ở mức cơ bản; có thể bổ sung lịch giao, khu vực phục vụ, người phụ trách và ghi chú xử lý đơn.

- **Báo cáo kinh doanh:** dashboard hiện cho cái nhìn tổng quan; phần phân tích doanh thu, chi phí, lợi nhuận, tồn kho và hiệu quả khuyến mãi theo thời gian còn có thể trình bày sâu hơn.
- **Trải nghiệm quản trị:** một số màn hình quản trị có nhiều chức năng; nên thống nhất lại cách lọc, tìm kiếm, phân trang và thao tác nhanh để nhân viên sử dụng thuận tiện hơn.
- **Nội dung tư vấn:** chatbot đã hỗ trợ hỏi đáp cơ bản; chất lượng câu trả lời sẽ tốt hơn khi FAQ, dữ liệu sản phẩm và kịch bản tư vấn được bổ sung thường xuyên từ quá trình vận hành thực tế.

6.4 Hướng Phát Triển

- Tích hợp thanh toán trực tuyến, đối soát giao dịch và hiển thị trạng thái thanh toán rõ ràng cho khách hàng.
- Hoàn thiện thông báo đơn hàng qua email, SMS hoặc kênh nội bộ để nhân viên xử lý đơn nhanh hơn.
- Mở rộng cấu hình giao hàng theo khung giờ, khu vực phục vụ, phí theo thời điểm và lựa chọn đơn vị vận chuyển.
- Phát triển báo cáo quản trị theo ngày, tuần, tháng, nhóm sản phẩm, khách hàng và hiệu quả chương trình khuyến mãi.
- Cải thiện chatbot theo hướng ghi nhận câu hỏi phổ biến, gợi ý sản phẩm chính xác hơn và hỗ trợ nhân viên bổ sung FAQ từ trang quản trị.
- Cải thiện phiên bản mobile và nội dung SEO để website dùng tốt hơn trong bối cảnh bán hàng thật.

6.5 Kiến Thức và Kinh Nghiệm Thu Được

Quá trình thực hiện giúp sinh viên củng cố cách xây dựng một hệ thống thương mại điện tử theo kiến trúc headless commerce, cách tách giao diện khách hàng khỏi backend, cách tổ chức dữ liệu sản phẩm–đơn hàng–tồn kho và cách mở rộng Medusa bằng các mô-đun riêng. Đề tài cũng là dịp luyện kỹ năng đọc mã nguồn lớn, đối chiếu giữa giao diện và nghiệp vụ, vẽ sơ đồ phân tích–thiết kế, cũng như trình bày kết quả thành một báo cáo có cấu trúc.

Một kinh nghiệm quan trọng là chức năng bán hàng không chỉ nằm ở trang sản phẩm hay nút đặt mua. Để tạo được một luồng mua hàng hoàn chỉnh, hệ thống phải phối hợp nhiều phần: dữ liệu sản phẩm, giá bán, tồn kho, phí giao hàng, khuyến mãi, tài khoản khách hàng, trạng thái

đơn và công cụ cho nhân viên vận hành.

6.6 Kết Luận

Mạng Fruitboxz đã triển khai được các thành phần chính của một website bán hàng cho cả khách hàng và cửa hàng. Hệ thống thể hiện được đặc thù của sản phẩm trái cây qua biến thể, hộp quà, giao hàng theo khu vực, chatbot tư vấn và tồn kho theo nguyên liệu. Dù vẫn còn các phần cần hoàn thiện như thanh toán trực tuyến, thông báo sau đặt hàng và báo cáo chuyên sâu, phiên bản hiện tại vẫn đủ để làm nền cho các bước phát triển tiếp theo của hệ thống.

6.7 Tài Liệu Tham Khảo

1. Medusa Documentation. <https://docs.medusajs.com/>
2. React Documentation. <https://react.dev/>
3. Vite Documentation. <https://vite.dev/guide/>
4. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
5. PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/15/>
6. Redis Documentation. <https://redis.io/docs/latest/>
7. Meilisearch Documentation. <https://www.meilisearch.com/docs/>
8. MinIO Documentation. <https://min.io/docs/minio/>
9. Docker Compose Documentation. <https://docs.docker.com/compose/>
10. Groq API Documentation. <https://console.groq.com/docs/>
11. Mã nguồn Mạng Fruitboxz. <https://github.com/pathust/mong.fruitboxz>